

Docker & DevOps: The Complete Guide

Comprehensive Course Notes — Building, Shipping & Deploying Containerized Applications

Course Notes

2026

Contents

Preface	8
Chapter 1: Introduction to DevOps and Containerization	9
1.1 What Is DevOps?	9
1.1.1 Core DevOps Principles	9
1.1.2 The DevOps Toolchain	9
1.2 The Problem Docker Solves	10
1.3 Virtual Machines vs. Containers	10
1.3.1 Virtual Machine Architecture	10
1.3.2 Container Architecture	10
1.3.3 Side-by-Side Comparison	11
1.3.4 Why the Shared Kernel Matters	11
1.4 What Exactly Is a Container?	12
1.5 Benefits of Containerization	12
1.6 A Brief History and the Role of Docker Inc.	13
1.7 What This Course Covers	13
Chapter 2: Docker Architecture	14
2.1 Client-Server Architecture	14
2.2 The Docker Daemon’s Responsibilities	14
2.3 Docker Objects	15
2.4 Underlying Technology: containerd and runc	15
2.5 Images Are Built From Layers	16
2.6 Docker Desktop vs. Docker Engine	16
Chapter 3: Installing Docker and Getting Started	18
3.1 Installation Options Overview	18
3.2 Installing on Linux (Ubuntu example)	18
3.2.1 Running Docker Without <code>sudo</code>	18
3.2.2 Enabling Docker to Start on Boot	19
3.3 Installing Docker Desktop on macOS	19
3.4 Installing Docker Desktop on Windows	19

3.5 Verifying and Exploring Your Installation	20
3.6 Docker Desktop GUI Overview	20
3.7 Docker Playground / Labs	20
Chapter 4: The Docker Command-Line Toolkit	21
4.1 <code>docker run</code> — Creating and Starting Containers	21
4.1.1 Basic Examples	21
4.1.2 Key Flags for <code>docker run</code>	21
4.1.3 Worked Example: Running an Nginx Web Server	22
4.2 Inspecting Container State	22
4.3 Container Lifecycle Commands	22
4.4 Logs and Debugging	22
4.4.1 <code>docker exec</code> — Running Commands in a Running Container	23
4.5 Copying Files Between Host and Container	23
4.6 Working with Images	23
4.7 System Cleanup	23
4.8 Hands-On Lab: Container Basics	24
4.9 Common Pitfalls	24
Chapter 5: Environment Variables and Container Configuration	25
5.1 Why Environment Variables Matter	25
5.2 Setting Environment Variables at Runtime	25
5.2.1 Loading from a File	25
5.3 Setting Default Environment Variables in a Dockerfile	25
5.4 Viewing a Container’s Environment Variables	26
5.5 Worked Example: A Configurable Database Container	26
5.6 Environment Variables in Docker Compose	26
5.7 Best Practices	26
Chapter 6: Docker Images and the Dockerfile	27
6.1 What Is an Image, Precisely?	27
6.2 The Dockerfile	27
6.2.1 Core Instructions	27
6.2.2 Full Instruction Reference	28
6.3 Building an Image	28
6.4 The <code>.dockerignore</code> File	29
6.5 Layer Caching in Depth	29
6.5.1 Anti-pattern (slow rebuilds)	29
6.5.2 Best practice (fast rebuilds)	29
6.6 Multi-Stage Builds	30
6.7 Choosing a Base Image	30
6.8 <code>CMD</code> vs <code>ENTRYPOINT</code>	31
6.8.1 The Two Forms	31
6.8.2 <code>CMD</code> Alone	31
6.8.3 <code>ENTRYPOINT</code> Alone	31
6.8.4 Combining <code>ENTRYPOINT</code> and <code>CMD</code> (the recommended pattern)	32
6.8.5 Summary Table	32
6.9 <code>HEALTHCHECK</code>	32

6.10 Image Naming, Tagging, and Versioning	33
Chapter 7: Docker Networking	34
7.1 Why Networking Matters	34
7.2 Network Drivers	34
7.3 The Default Bridge Network	34
7.4 User-Defined Bridge Networks (Recommended)	35
7.5 The <code>host</code> Network	35
7.6 The <code>none</code> Network	35
7.7 Publishing Ports	35
7.8 Inspecting Network Configuration	36
7.9 Container-to-Container Communication: Worked Example	36
7.10 DNS and Service Discovery Summary	36
7.11 Common Pitfalls	37
Chapter 8: Docker Storage — Volumes, Bind Mounts, and tmpfs	38
8.1 Why Persistent Storage Is Necessary	38
8.2 The Three Storage Mechanisms	38
8.3 Volumes	38
8.3.1 Creating and Using Volumes	38
8.3.2 Anonymous Volumes	39
8.3.3 Sharing a Volume Across Multiple Containers	39
8.4 Bind Mounts	39
8.4.1 The Modern <code>--mount</code> Syntax	39
8.4.2 Bind Mounts for Local Development	40
8.5 tmpfs Mounts (Linux Only)	40
8.6 Volume Drivers and Cloud Storage	40
8.7 Backing Up and Restoring a Volume	40
8.8 Choosing Between Volumes and Bind Mounts	41
Chapter 9: Docker Compose — Multi-Container Applications	42
9.1 The Problem Compose Solves	42
9.2 Installing Compose	42
9.3 Anatomy of a <code>docker-compose.yml</code> File	42
9.3.1 Key Sections Explained	43
9.4 Core Compose Commands	43
9.5 Scaling Services	44
9.6 Multiple Compose Files and Overrides	44
9.7 <code>depends_on</code> and Startup Ordering	44
9.8 Environment Variable Substitution	45
9.9 Worked Example: A Full-Stack App (Frontend, API, Database, Cache)	45
9.10 Compose vs. Swarm vs. Kubernetes	46
Chapter 9b: Advanced Docker Compose Features	47
9b.1 Profiles — Selectively Enabling Services	47
9b.2 <code>extends</code> and YAML Anchors for Reuse	47
9b.3 Resource Limits in Compose	48
9b.4 <code>docker compose watch</code> — Live Development Sync	48

9b.5 Dependency Conditions Beyond <code>service_healthy</code>	48
9b.6 Named Compose Projects	49
Chapter 10: Docker Registries and Image Distribution	50
10.1 What Is a Registry?	50
10.2 Docker Hub	50
10.2.1 Authenticating	50
10.3 Pushing and Pulling Images	50
10.4 Running a Private Registry	51
10.5 Image Naming Conventions for Registries	51
10.6 Image Digests and Immutability	51
10.7 Vulnerability Scanning	52
10.8 Rate Limits and Caching	52
Chapter 11: The Docker Engine, Deep Dive	53
11.1 Recap and Expansion	53
11.2 The Full Component Stack	53
11.3 BuildKit	53
11.4 Storage Drivers	54
11.5 The Container Runtime Spec (OCI)	54
11.6 Daemon Configuration	54
11.7 Logging Drivers	54
11.8 Resource Isolation and Limits Revisited	55
11.9 Security Features of the Engine	55
Chapter 11b: Beyond Docker — The Broader Container Ecosystem	56
11b.1 Podman	56
11b.2 containerd and nerdctl	56
11b.3 Buildah and Skopeo	56
11b.4 LXC/LXD	56
11b.5 gVisor and Kata Containers — Stronger Isolation Runtimes	57
11b.6 Where Docker Fits Today	57
Chapter 12: Docker on Windows and macOS	58
12.1 Why This Needs Its Own Chapter	58
12.2 Docker Desktop on macOS	58
12.2.1 Practical Implications	58
12.3 Docker Desktop on Windows	58
12.3.1 WSL2 Backend (Recommended)	58
12.3.2 Hyper-V Backend (Legacy)	59
12.3.3 Windows Containers (Distinct From Linux Containers)	59
12.4 Common Cross-Platform Gotchas	59
12.5 When to Just Use Linux	60
Chapter 13: Container Orchestration — Why and What	61
13.1 From One Container to a Fleet	61
13.2 What an Orchestrator Actually Does	61
13.3 The Two Major Orchestrators Covered in This Course	62

13.4 A Cluster's Building Blocks (Shared Vocabulary)	62
Chapter 14: Docker Swarm	63
14.1 What Is Swarm?	63
14.2 Initializing a Swarm	63
14.3 Manager Nodes and Raft Consensus	63
14.4 Services — The Core Swarm Abstraction	63
14.4.1 Scaling a Service	64
14.4.2 Rolling Updates	64
14.5 The Routing Mesh and Load Balancing	64
14.6 Stacks — Deploying Compose Files to Swarm	65
14.7 Secrets and Configs	66
14.8 Placement Constraints and Node Labels	66
14.9 When Swarm Is a Good Fit	66
Chapter 15: Kubernetes Fundamentals	68
15.1 What Is Kubernetes and Why Learn It Here	68
15.2 Kubernetes Architecture	68
15.2.1 Control Plane Components	68
15.2.2 Node (Worker) Components	68
15.3 The Pod — Kubernetes' Smallest Deployable Unit	69
15.4 Deployments — Managing Replicated, Self-Healing Pods	70
15.4.1 Readiness vs. Liveness Probes	71
15.5 Services — Stable Networking for Ephemeral Pods	71
15.5.1 Service Types	71
15.6 Ingress — HTTP(S) Routing Into the Cluster	72
15.7 ConfigMaps and Secrets	73
15.8 Namespaces	73
15.9 Persistent Storage	73
15.10 <code>kubectl</code> Quick Reference	74
15.11 Helm — Packaging Kubernetes Applications	74
15.12 Local Kubernetes for Development and Learning	75
15.13 Docker Swarm vs. Kubernetes — Summary Comparison	75
Chapter 16: CI/CD and Deploying Containerized Applications	76
16.1 The Full Pipeline, End to End	76
16.2 Continuous Integration: Building and Testing the Image	76
16.3 Continuous Deployment: Rolling Out the New Image	77
16.3.1 Deploying to Docker Swarm	77
16.3.2 Deploying to Kubernetes	77
16.3.3 Deploying to a Single Server (Simple Cases)	77
16.4 Deployment Strategies	77
16.5 Health Checks Tie the Whole Pipeline Together	78
16.6 Observability in Production	78
Chapter 16b: Capstone Project — Containerizing and Deploying a Three-Tier Application	79
16b.1 Project Layout	79

16b.2 The API Dockerfile	79
16b.3 The Frontend Dockerfile	80
16b.4 Local Development with Compose	80
16b.5 CI: Build, Test, Push	81
16b.6 Production Deployment: Docker Swarm Stack	82
16b.7 What This Project Demonstrates	83
Chapter 17: Best Practices, Security, and Common Pitfalls	84
17.1 Dockerfile Best Practices Checklist	84
17.2 Image Security	84
17.2.1 Minimize the Attack Surface	84
17.2.2 Scan Images for Known Vulnerabilities	84
17.2.3 Don't Run Containers as Root Unnecessarily	85
17.2.4 Avoid <code>--privileged</code> and Unnecessary Capabilities	85
17.2.5 Read-Only Root Filesystems	85
17.2.6 Keep the Docker Engine and Host OS Patched	85
17.3 Networking and Secrets Hygiene	85
17.4 Operational Best Practices	85
17.5 Common Beginner Mistakes, Revisited	86
Chapter 17b: Troubleshooting Docker — A Practical Field Guide	87
17b.1 “Cannot connect to the Docker daemon”	87
17b.2 Container Exits Immediately After <code>docker run</code>	87
17b.3 “Port is already allocated”	87
17b.4 Container Can't Reach Another Container by Name	88
17b.5 “No space left on device”	88
17b.6 Build Is Slow or Cache Never Hits	88
17b.7 “Permission denied” Inside a Container	88
17b.8 Image Builds Fine Locally but Fails in CI	89
17b.9 Container Runs Out of Memory (OOMKilled)	89
17b.10 Swarm/Kubernetes Deployment Stuck (“Pending” or Not Updating)	89
Chapter 18: Conclusion and Next Steps	90
18.1 What You've Learned	90
18.2 Suggested Learning Path Going Forward	90
18.3 Closing Thought	91
Appendix A: Complete Command Cheat Sheet	92
A.1 Container Lifecycle	92
A.2 Inspection and Debugging	92
A.3 Images	92
A.4 Networks	92
A.5 Volumes	92
A.6 Cleanup	93
A.7 Compose	93
A.8 Swarm	93
A.9 Kubernetes (<code>kubectl</code>)	93

Appendix B: Dockerfile Instruction Reference	94
Appendix C: Glossary	95
Appendix E: Further Resources	97
E.1 Official Documentation	97
E.2 Hands-On Practice	97
E.3 Reference Images and Examples	97
E.4 Security	97
E.5 Suggested Practice Progression	97
Appendix D: Review Questions and Answers	98
D.1 Chapter 1 — Introduction to DevOps and Containerization	98
D.2 Chapter 2 — Docker Architecture	98
D.3 Chapter 3 — Installing Docker	98
D.4 Chapter 4 — The Docker CLI	98
D.5 Chapter 5 — Environment Variables	99
D.6 Chapter 6 — Images and the Dockerfile	99
D.7 Chapter 7 — Networking	99
D.8 Chapter 8 — Storage	99
D.9 Chapter 9 — Docker Compose	100
D.10 Chapter 10 — Registries	100
D.11 Chapter 11 — The Docker Engine	100
D.12 Chapter 12 — Windows and macOS	100
D.13 Chapter 13 — Orchestration Overview	100
D.14 Chapter 14 — Docker Swarm	100
D.15 Chapter 15 — Kubernetes	101
D.16 Chapter 16 — CI/CD and Deployment	101
D.17 Chapter 17 — Best Practices and Security	101

Preface

These notes are a comprehensive, self-contained companion to a full-length Docker and DevOps video course covering containerization from first principles through to production deployment and orchestration. They are organized to mirror the structure of the course:

1. Foundations — what containers are and why they matter
2. Installing and getting started with Docker
3. The Docker command-line toolkit
4. Images, Dockerfiles, and the build process
5. Networking
6. Storage and persistence
7. Docker Compose for multi-container applications
8. Registries and image distribution
9. The Docker Engine internals
10. Platform-specific notes (Windows/Mac)
11. Orchestration — Docker Swarm
12. Orchestration — Kubernetes fundamentals
13. CI/CD and deploying containerized apps
14. Best practices and security
15. Appendices — command cheat sheets, Dockerfile reference, glossary

Each chapter combines conceptual explanation with hands-on command references, worked examples, diagrams described in text, and common pitfalls. Code blocks show exact commands you can run along with expected behavior. Treat this as both a first read and a reference you return to.

Chapter 1: Introduction to DevOps and Containerization

1.1 What Is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity. It combines software **D**evelopment and IT **O**perations, which historically operated as separate, often siloed teams with conflicting incentives:

- **Developers** are incentivized to ship new features quickly.
- **Operations** teams are incentivized to keep systems stable, which historically meant resisting change.

DevOps dissolves this wall by making a single team (or tightly collaborating teams) responsible for the entire lifecycle of an application: planning, coding, building, testing, releasing, deploying, operating, and monitoring. The goal is to shorten the development lifecycle while delivering features, fixes, and updates frequently and reliably.

1.1.1 Core DevOps Principles

- **Automation** — manual, error-prone steps (builds, tests, deployments, provisioning) are replaced with scripted, repeatable pipelines.
- **Continuous Integration (CI)** — developers merge code changes into a shared repository frequently, with each merge automatically built and tested.
- **Continuous Delivery / Deployment (CD)** — validated code is automatically prepared for release (Delivery) or automatically pushed to production (Deployment).
- **Infrastructure as Code (IaC)** — infrastructure (servers, networks, load balancers) is defined in version-controlled configuration files rather than provisioned by hand.
- **Monitoring and Observability** — systems emit logs, metrics, and traces so teams can detect and diagnose problems quickly.
- **Collaboration and shared ownership** — “you build it, you run it.”

1.1.2 The DevOps Toolchain

A typical DevOps toolchain touches every phase of delivery:

Phase	Example Tools
Plan	Jira, Trello, Azure Boards
Code	Git, GitHub, GitLab, Bitbucket
Build	Maven, Gradle, npm, Docker
Test	JUnit, Selenium, pytest
Release/Deploy	Jenkins, GitHub Actions, GitLab CI, Azure DevOps, ArgoCD
Operate	Kubernetes, Ansible, Terraform
Monitor	Prometheus, Grafana, ELK Stack, Datadog

Docker sits at the heart of the **Build**, **Release**, and **Operate** phases: it standardizes how software is packaged and how it runs, which is precisely the problem containerization was invented to solve.

1.2 The Problem Docker Solves

Before containers, deploying an application typically meant provisioning a physical or virtual machine, installing an operating system, then manually installing the application’s runtime (say, a specific version of Node.js, Python, or Java), its system-level dependencies (libraries, drivers), and finally the application code itself.

This created what is famously known as the “**it works on my machine**” problem:

- A developer builds and tests an application on their laptop, where a particular version of a language runtime and set of libraries happen to be installed.
- The application is deployed to a test server, which has slightly different versions of those dependencies.
- The application behaves differently — or crashes outright — because of the environment mismatch.
- The same problem repeats between test and production environments, and between production servers when scaling horizontally.

Traditional solutions to this problem were heavyweight:

- **Detailed setup documentation** — brittle, quickly goes stale, and requires manual execution.
- **Virtual Machines (VMs)** — each VM bundles a full guest operating system along with the application, guaranteeing a consistent environment, but at a steep cost in disk space, memory, and boot time.

1.3 Virtual Machines vs. Containers

Understanding the distinction between VMs and containers is foundational to understanding why Docker exists.

1.3.1 Virtual Machine Architecture

A virtual machine emulates an entire physical computer. The stack looks like this, from bottom to top:

1. **Physical Host Hardware** — CPU, RAM, disk, network interfaces.
2. **Host Operating System** (optional, depending on hypervisor type).
3. **Hypervisor** — software (e.g., VMware ESXi, Oracle VirtualBox, Microsoft Hyper-V, KVM) that creates and manages virtual machines, allocating physical resources to each.
4. **Guest Operating System** — each VM runs its own, complete operating system (its own kernel, drivers, and system libraries), entirely independent of the host OS.
5. **Application and its dependencies** — installed inside the guest OS.

Because each VM carries a full OS, a single VM image is often several gigabytes, and booting a VM means booting an entire operating system — commonly taking anywhere from tens of seconds to a few minutes.

1.3.2 Container Architecture

A container, by contrast, does **not** bundle a guest operating system. Instead, containers share the **host machine’s OS kernel** and isolate the application’s processes from everything else using

kernel features. The stack looks like this:

1. **Physical Host Hardware**
2. **Host Operating System** — a single shared kernel (typically Linux)
3. **Container Runtime** (e.g., Docker Engine, containerd) — manages container lifecycles
4. **Containers** — each is an isolated user-space instance containing only the application and the specific libraries/dependencies it needs, nothing more

Because containers don't carry a full OS, container images are typically measured in megabytes rather than gigabytes, and containers start in a fraction of a second to a few seconds, since there's no OS boot process — the container runtime simply starts the application process with an isolated view of the filesystem, network, and process table.

1.3.3 Side-by-Side Comparison

Aspect	Virtual Machines	Containers
Isolation level	Full hardware-level virtualization	OS-level (process) virtualization
What's bundled	Full guest OS + app + dependencies	App + dependencies only
Typical image size	Gigabytes	Megabytes
Startup time	Seconds to minutes	Milliseconds to seconds
Kernel	Each VM has its own kernel	Containers share the host kernel
Resource overhead	High (each VM duplicates OS overhead)	Low (near-native performance)
Density per host	Fewer VMs per host	Many more containers per host
Portability	Portable, but heavy to move	Highly portable, lightweight to move
Security boundary	Very strong (separate kernels)	Weaker than VMs by default (shared kernel), but strong with proper configuration

1.3.4 Why the Shared Kernel Matters

Because containers share the host's kernel, a Linux container image can only run on a host with a compatible Linux kernel (this is why “Docker on Windows/Mac” historically runs a lightweight Linux VM under the hood — more on this in Chapter 10). This is the fundamental trade-off: containers give up the total isolation of a separate kernel in exchange for dramatically lower overhead and faster startup.

In practice, for the vast majority of application workloads (web servers, APIs, background workers, databases), this trade-off is well worth it, and the isolation containers provide via kernel namespaces and control groups (cgroups) is sufficient. For workloads requiring the strongest possible isolation (e.g., running fully untrusted multi-tenant code), VMs — or “micro-VM” technologies like Firecracker that combine ideas from both — remain relevant.

1.4 What Exactly Is a Container?

A container is a **standardized, self-contained unit of software** that packages up application code together with all the libraries, binaries, configuration files, and dependencies it needs to run, so that it behaves identically regardless of the environment it's deployed to — a developer's laptop, a testing server, or a production cluster in the cloud.

Two ideas are bundled inside “container”:

1. **Container image** — an immutable, read-only template — a snapshot of a filesystem plus metadata (what command to run, environment variables, exposed ports, etc.). Images are built once and can be run many times.
2. **Container** — a running (or stopped) *instance* of an image, analogous to how an object is an instance of a class in object-oriented programming. You can start many containers from the same image.

Under the hood, Docker containers rely on two core Linux kernel features:

- **Namespaces** — provide isolation. Each container gets its own view of process IDs (PID namespace), network interfaces (network namespace), mount points (mount namespace), host-name (UTS namespace), inter-process communication (IPC namespace), and user IDs (user namespace). From inside a container, it looks like the container has the entire machine to itself.
- **Control groups (cgroups)** — provide resource limiting and accounting. Cgroups let the runtime cap how much CPU, memory, disk I/O, and network bandwidth a container (or group of containers) can consume, preventing any one container from starving the others.

Docker did not invent namespaces or cgroups — these existed in the Linux kernel already (and technologies like LXC/LXD used them before Docker). Docker's innovation was building a friendly, standardized toolchain and image format (later standardized as the **OCI — Open Container Initiative — image and runtime specifications**) around these primitives, plus a registry ecosystem (Docker Hub) for sharing images, which drove containers into mainstream adoption starting around 2013.

1.5 Benefits of Containerization

- **Consistency across environments** — “build once, run anywhere.” The same image runs identically on a laptop, CI server, or production cluster.
- **Isolation** — each container has its own filesystem, process space, and network stack, preventing dependency conflicts between applications on the same host.
- **Lightweight and fast** — low overhead compared to VMs means higher density and faster startup, which is essential for auto-scaling and rapid iteration.
- **Version control for environments** — a Dockerfile is a text file checked into source control, meaning the exact environment an application needs is versioned alongside the code.
- **Microservices enablement** — containers make it practical to decompose a monolithic application into many small, independently deployable services, each in its own container.
- **Efficient resource utilization** — many containers can share a single host's resources far more efficiently than many VMs.
- **Simplified CI/CD** — the same container image that passes automated tests is the exact artifact promoted through staging and into production, eliminating “environment drift.”

1.6 A Brief History and the Role of Docker Inc.

- **2008** — Linux kernel gains full **cgroups** support, and **LXC (Linux Containers)** emerges as an early full-system container manager built on namespaces and cgroups.
- **2013** — Docker, Inc. (originally dotCloud) releases Docker, initially built on top of LXC, and later replaces LXC with its own library, **libcontainer**. Docker’s contribution was developer experience: a simple CLI, a standard image format, layered filesystems for efficient image storage/distribution, and Docker Hub as a public registry.
- **2015** — The **Open Container Initiative (OCI)** is founded to standardize container image formats and runtimes, so that images and runtimes from different vendors remain interoperable.
- **2017 onward** — Docker’s internals are broken into smaller, composable pieces — **containerd** (a container runtime, donated to the Cloud Native Computing Foundation) and **runc** (a low-level OCI-compliant runtime for actually spawning containers) — which are now used not just by Docker but by Kubernetes and other platforms directly.
- **Present day** — “Docker” commonly refers both to Docker Inc.’s product suite (Docker Desktop, Docker Engine, Docker Hub) and, informally, to the broader OCI-compatible container ecosystem. Kubernetes has become the dominant *orchestrator* for running containers at scale, often using containerd directly as its runtime rather than the full Docker Engine — a distinction covered in Chapter 12.

1.7 What This Course Covers

By the end of these notes, you will understand:

- How to install Docker and run your first containers
- The full Docker CLI surface for managing containers, images, networks, and volumes
- How to write Dockerfiles to build custom images, including multi-stage builds
- Networking models for connecting containers to each other and the outside world
- Persisting data with volumes and bind mounts
- Defining and running multi-container applications with Docker Compose
- Distributing images through registries (Docker Hub and private registries)
- The architecture of the Docker Engine itself
- Platform differences on Windows and macOS
- Orchestrating containers at scale with Docker Swarm
- The fundamentals of Kubernetes as the industry-standard orchestrator
- How containerized applications fit into a full CI/CD pipeline and get deployed to production

Chapter 2: Docker Architecture

2.1 Client-Server Architecture

Docker uses a **client-server architecture**. Understanding this is essential, because it explains why Docker commands behave the way they do, and it sets up everything that follows about the Docker Engine (Chapter 9).

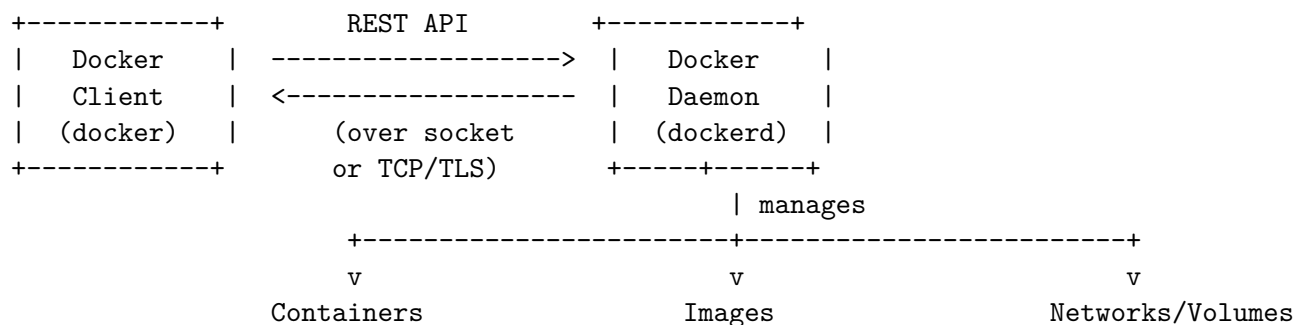
The three major components are:

1. **Docker Client (`docker`)** — the command-line tool (and any GUI, like Docker Desktop) that users interact with directly. When you type `docker run`, `docker build`, or `docker ps`, you are using the client.
2. **Docker Daemon (`dockerd`)** — a background service (a “daemon”) that does the actual work: building images, running containers, managing networks and volumes. The daemon listens for API requests and manages Docker objects.
3. **Docker Registry** — a storage and distribution system for Docker images. **Docker Hub** is the default public registry, but organizations commonly run private registries (Chapter 8).

The client and daemon communicate over a **REST API**, which can happen:

- **Locally**, over a Unix socket (`/var/run/docker.sock` on Linux/Mac) — the default and most common setup.
- **Remotely**, over a network interface using TCP, optionally secured with TLS certificates, allowing a local `docker` client to control a daemon running on a different machine.

This separation matters practically: it’s why you can run `docker` commands from your laptop against a Docker Engine running on a remote server, and it’s why Docker Desktop on Mac/Windows works the way it does (the client talks to a daemon running inside a hidden Linux VM — see Chapter 10).



2.2 The Docker Daemon’s Responsibilities

The daemon (`dockerd`) is responsible for the full lifecycle of Docker objects:

- Building images from Dockerfiles
- Pulling images from and pushing images to registries
- Creating, starting, stopping, and removing containers
- Creating and managing networks that containers attach to
- Creating and managing volumes for persistent storage
- Exposing a REST API that the client (or any other tool) can use

The daemon can also communicate with other daemons to manage **Docker Swarm** services (Chapter 11), coordinating container placement across a cluster of machines.

2.3 Docker Objects

When you use Docker, you are creating and managing several kinds of objects:

Object	Description
Image	Read-only template with instructions for creating a container. Built in layers.
Container	A runnable instance of an image. Can be started, stopped, moved, deleted.
Network	Enables communication between containers, and between containers and the outside world.
Volume	A mechanism for persisting data generated and used by containers, independent of the container lifecycle.
Plugin	Extends Docker’s functionality (e.g., third-party volume or network drivers).

2.4 Underlying Technology: containerd and runc

As of Docker’s modern architecture, `dockerd` does not directly create containers. Instead there is a layered chain of responsibility:

```
docker (CLI)
  |
  v
dockerd (Docker daemon) -- handles images, networks, volumes, build, API
  |
  v
containerd -- a high-level container runtime; manages container lifecycle,
              image pulls/pushes/storage, supervises execution
  |
  v
containerd-shim -- a small process that sits between containerd and the
                  actual container process, allowing containerd to be
                  restarted without killing running containers
  |
  v
runc -- a low-level, OCI-compliant runtime that does the actual work of
        creating namespaces/cgroups and starting the container process
```

- **containerd** was originally part of Docker and was donated to the **Cloud Native Computing Foundation (CNCF)** in 2017. It is now a graduated CNCF project used directly by Kubernetes (via the Container Runtime Interface, CRI) as well as by Docker.
- **runc** implements the **OCI Runtime Specification** — the industry-standard, low-level contract for “how to start a container,” which was itself derived from Docker’s original `libcontainer`.

This layered design means that “Docker” is really a user-friendly toolkit built atop standardized, swappable, industry-shared components — which is why Kubernetes can run containers without needing the full Docker Engine at all, using containerd (or another OCI-compliant runtime like CRI-O) directly.

2.5 Images Are Built From Layers

Docker images are composed of **read-only layers**, stacked on top of one another. Each instruction in a Dockerfile (Chapter 4) typically produces one layer.

- Layers are cached and content-addressed (identified by a hash of their contents).
- If two images share a common ancestor layer (e.g., they’re both built **FROM ubuntu:22.04**), Docker stores that layer only **once** on disk and reuses it for both images.
- When you rebuild an image, layers that haven’t changed are reused from cache, and only layers after the first changed instruction are rebuilt — this is the basis of Docker’s build caching, which dramatically speeds up iterative development.

When a container is created from an image, Docker adds one more layer on top: a thin, writable **container layer**. All changes made while the container runs (new files, modified files) are written to this layer using a **copy-on-write (CoW)** strategy — if a container modifies a file that exists in a read-only image layer below it, that file is first copied up into the writable container layer, then modified there, leaving the original image layer untouched.

```
+-----+
|   Writable Container Layer   | ← created per-container; disappears
+-----+                     when the container is removed
| Image Layer: COPY app code  |
+-----+
| Image Layer: RUN pip install | ← read-only, shared across
+-----+                     any container using this image
| Image Layer: FROM python:3.12|
+-----+
```

This is why containers are so lightweight to start: starting a new container from an existing image only requires creating a new, empty writable layer — no copying of the entire filesystem is needed.

It also explains an important characteristic of containers: **any data written to the writable container layer is lost when the container is removed**, unless it is explicitly persisted using volumes or bind mounts (Chapter 6).

2.6 Docker Desktop vs. Docker Engine

It’s worth disambiguating commonly confused terms:

- **Docker Engine** — the core, open-source, client-server technology described above (**dockerd**, the CLI, the API). This is what actually runs containers. It is native to Linux.
- **Docker Desktop** — a commercial application (free for personal use and small businesses, licensed for larger enterprises) for Windows and macOS that bundles: a Linux virtual machine to host the Docker Engine (since containers require a Linux kernel), the Docker CLI, Docker Compose, Kubernetes (optional, single-node), and a graphical UI for managing containers, images, and volumes.

- **Docker Hub** — the default public registry/cloud service for storing and sharing images.
- **Docker Compose** — a tool (and file format) for defining and running multi-container applications, covered fully in Chapter 7.

Chapter 3: Installing Docker and Getting Started

3.1 Installation Options Overview

Platform	Recommended Install	Notes
Linux	Docker Engine (native)	Runs natively; no VM needed
macOS	Docker Desktop	Runs a lightweight Linux VM under the hood
Windows	Docker Desktop (with WSL2 backend)	Uses WSL2's Linux kernel
Cloud servers	Docker Engine via install script or package manager	e.g., AWS EC2, DigitalOcean droplets

3.2 Installing on Linux (Ubuntu example)

Linux is the “native” home of Docker since containers rely directly on the Linux kernel — no virtualization layer is required.

```
# 1. Remove any old versions
sudo apt-get remove docker docker-engine docker.io containerd runc

# 2. Set up Docker's official apt repository
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 3. Install Docker Engine, CLI, containerd, and plugins
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# 4. Verify installation
sudo docker run hello-world
```

3.2.1 Running Docker Without sudo

By default, the Docker daemon binds to a Unix socket owned by **root**, meaning every **docker** command must be prefixed with **sudo** unless your user is added to the **docker** group:

```

sudo groupadd docker          # create the group if it doesn't exist
sudo usermod -aG docker $USER # add current user to the docker group
newgrp docker                 # activate the group change in current shell
docker run hello-world        # now works without sudo

```

Security note: Membership in the `docker` group is effectively equivalent to root access on the host, because a container can be started with a bind mount of the entire host filesystem. Only trusted users should be added to this group.

3.2.2 Enabling Docker to Start on Boot

```

sudo systemctl enable docker.service
sudo systemctl enable containerd.service
sudo systemctl status docker      # check daemon status

```

3.3 Installing Docker Desktop on macOS

1. Download the installer from Docker’s official site (choose Intel chip or Apple Silicon build as appropriate).
2. Drag `Docker.app` into `/Applications`.
3. Launch Docker Desktop; it will request permission to install its helper components and start the Linux VM.
4. Verify from a terminal:

```

docker --version
docker run hello-world

```

Docker Desktop on Mac runs a purpose-built lightweight Linux VM (historically using HyperKit, more recently an internal virtualization framework leveraging Apple’s `Virtualization.framework`) to host the actual Linux-based Docker Engine, since macOS’s kernel (XNU) cannot run Linux containers natively.

3.4 Installing Docker Desktop on Windows

1. **Enable WSL2** (Windows Subsystem for Linux, version 2), Docker Desktop’s recommended backend:

```
wsl --install
```

2. Download and run the Docker Desktop installer, ensuring “Use WSL 2 instead of Hyper-V” is selected during setup (this is the default on modern Windows 10/11).
3. Restart when prompted.
4. Launch Docker Desktop and verify from PowerShell or a WSL2 terminal:

```

docker --version
docker run hello-world

```

With the WSL2 backend, containers run inside the real Linux kernel provided by WSL2, giving near-native Linux container performance — a significant improvement over the older Hyper-V-based backend, which ran a heavier, more traditional VM.

3.5 Verifying and Exploring Your Installation

```
docker version      # client and server (daemon) version info
docker info         # detailed system-wide information: containers,
                    # images, storage driver, cgroup version, etc.
docker system df     # disk usage by images, containers, volumes
```

Running `docker run hello-world` is the canonical “smoke test”: Docker will (1) look for the `hello-world` image locally, (2) not find it, (3) pull it from Docker Hub, (4) create a container from it, (5) run it — printing a confirmation message — and (6) the container exits immediately since its job is done.

3.6 Docker Desktop GUI Overview

Docker Desktop provides a graphical dashboard for:

- Browsing running and stopped **Containers**, with one-click start/stop/delete and log viewing
- Browsing locally stored **Images**, with size and layer information
- Browsing **Volumes**
- A built-in terminal to `exec` into running containers
- Settings for resource limits (CPU, memory, disk allocated to the underlying VM)
- Optional single-node **Kubernetes** cluster you can enable with one click

While the GUI is convenient for exploration, production workflows and automation always rely on the CLI, which is what the remainder of these notes focus on.

3.7 Docker Playground / Labs

For hands-on practice without installing anything locally, **Play with Docker** (labs.play-with-docker.com) provides free, temporary, browser-based Docker environments — useful for following along with labs, testing multi-node Swarm clusters, or trying destructive commands safely.

Chapter 4: The Docker Command-Line Toolkit

This chapter is the practical core of container management day-to-day: creating, inspecting, and controlling containers using the `docker` CLI. Commands are grouped by function, with syntax, common flags, and worked examples.

4.1 `docker run` — Creating and Starting Containers

`docker run` is the single most-used Docker command. It combines two operations: `docker create` (create a container from an image) and `docker start` (start it).

```
docker run [OPTIONS] IMAGE[:TAG] [COMMAND] [ARG...]
```

4.1.1 Basic Examples

```
docker run ubuntu           # run Ubuntu, execute default command, exit
docker run ubuntu sleep 5   # run Ubuntu, sleep 5 seconds, then exit
docker run -it ubuntu bash  # run interactively with a pseudo-TTY,
                             # dropping into a bash shell
docker run -d nginx         # run nginx in detached (background) mode
```

4.1.2 Key Flags for `docker run`

Flag	Purpose
<code>-d, --detach</code>	Run container in the background, print container ID
<code>-it</code>	Combine <code>-i</code> (interactive, keep STDIN open) and <code>-t</code> (allocate a pseudo-TTY) — used for interactive shells
<code>--name NAME</code>	Assign a human-readable name instead of a random one
<code>-p HOST:CONTAINER</code>	Publish (map) a container port to a host port
<code>-P</code>	Publish all exposed ports to random host ports
<code>-e KEY=VALUE</code>	Set an environment variable inside the container
<code>--env-file FILE</code>	Load environment variables from a file
<code>-v HOST:CONTAINER</code>	Bind mount / volume mount (Chapter 6)
<code>--rm</code>	Automatically remove the container when it exits
<code>--network NAME</code>	Attach the container to a specific network
<code>--restart POLICY</code>	Restart policy: <code>no</code> , <code>on-failure[:max-retries]</code> , <code>always</code> , <code>unless-stopped</code>
<code>-m, --memory</code>	Limit container memory (e.g., <code>-m 512m</code>)
<code>--cpus</code>	Limit CPU usage (e.g., <code>--cpus="1.5"</code>)
<code>-w, --workdir</code>	Set the working directory inside the container
<code>-u, --user</code>	Run as a specific user/UID instead of the image default (often root)

Flag	Purpose
<code>--entrypoint</code>	Override the image's ENTRYPOINT

4.1.3 Worked Example: Running an Nginx Web Server

```
docker run -d --name webserver -p 8080:80 nginx
```

This pulls `nginx` (if not present locally), starts it detached, names the container `webserver`, and maps host port 8080 to the container's port 80. Visiting `http://localhost:8080` now serves the default Nginx welcome page.

4.2 Inspecting Container State

```
docker ps                # list running containers
docker ps -a             # list ALL containers, including stopped ones
docker ps -q             # print only container IDs (useful for scripting)
docker ps --filter "status=exited" # filter by status
```

Typical `docker ps` columns: CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS, NAMES.

```
docker inspect CONTAINER # full JSON metadata: config, network settings,
                        # mounts, environment variables, etc.
docker inspect --format '{{.NetworkSettings.IPAddress}}' CONTAINER
docker top CONTAINER     # processes running inside the container
docker stats             # live stream of CPU/memory/network usage
                        # for all running containers
docker stats CONTAINER   # stream stats for one container
```

4.3 Container Lifecycle Commands

```
docker start CONTAINER # start a stopped container
docker stop CONTAINER  # gracefully stop (sends SIGTERM, then SIGKILL
                        # after a grace period, default 10s)
docker stop -t 30 CONTAINER # custom grace period (seconds)
docker restart CONTAINER   # stop then start
docker pause CONTAINER     # freeze all processes in the container (SIGSTOP)
docker unpause CONTAINER   # resume
docker kill CONTAINER      # send SIGKILL immediately (forceful stop)
docker rm CONTAINER        # remove a stopped container
docker rm -f CONTAINER     # force-remove a running container (stop + rm)
docker rm $(docker ps -aq) # remove ALL containers (running ones will
                        # error unless -f is added)
```

4.4 Logs and Debugging

```
docker logs CONTAINER # print all logs (stdout/stderr) so far
docker logs -f CONTAINER # follow (stream) logs live, like `tail -f`
```

```
docker logs --tail 100 CONTAINER # last 100 lines only
docker logs --since 10m CONTAINER # logs from the last 10 minutes
```

4.4.1 docker exec — Running Commands in a Running Container

```
docker exec CONTAINER COMMAND # run a one-off command
docker exec -it CONTAINER bash # open an interactive shell inside a
                                # running container - extremely
                                # common for debugging
docker exec -it CONTAINER sh   # use `sh` if `bash` isn't installed
                                # (common in minimal images like Alpine)
```

`docker exec` differs fundamentally from `docker run`: `run` creates a brand-new container from an image, while `exec` runs an additional process **inside an already-running container**.

4.5 Copying Files Between Host and Container

```
docker cp CONTAINER:/path/in/container ./local/path # container → host
docker cp ./local/file CONTAINER:/path/in/container # host → container
```

4.6 Working with Images

```
docker images # list locally stored images
docker image ls # equivalent, newer syntax
docker pull IMAGE[:TAG] # download an image from a registry
docker push IMAGE[:TAG] # upload an image to a registry
docker rmi IMAGE # remove a local image
docker image rm IMAGE # equivalent
docker image prune # remove unused (dangling) images
docker image prune -a # remove ALL unused images, not just
                      # dangling ones
docker history IMAGE # show the layer history of an image
docker tag SOURCE_IMAGE TARGET_IMAGE # create a new tag pointing at an
                                      # existing image
```

4.7 System Cleanup

Over time, stopped containers, dangling images, unused networks, and unused volumes accumulate and consume disk space.

```
docker system df # show disk usage summary
docker system prune # remove stopped containers, dangling
                    # images, unused networks (with confirmation)
docker system prune -a # also remove all unused images (not
                       # just dangling ones) - more aggressive
docker system prune -a --volumes # also remove unused volumes - use
                                  # with caution, this deletes data!
```

```
docker container prune      # remove only stopped containers
docker volume prune        # remove only unused volumes
docker network prune       # remove only unused networks
```

4.8 Hands-On Lab: Container Basics

A typical hands-on lab sequence for practicing the fundamentals:

```
# 1. Run an Ubuntu container interactively and explore
docker run -it ubuntu bash
# inside the container:
cat /etc/os-release
exit

# 2. Run a container in the background and check on it
docker run -d --name mynginx -p 8081:80 nginx
docker ps
curl http://localhost:8081

# 3. View logs and exec into it
docker logs mynginx
docker exec -it mynginx bash
  ls /usr/share/nginx/html
  exit

# 4. Stop, restart, and clean up
docker stop mynginx
docker ps -a
docker start mynginx
docker rm -f mynginx
docker ps -a
```

4.9 Common Pitfalls

- **Forgetting `-it`** and wondering why an interactive shell “immediately exits” — without a TTY and open STDIN, an interactive shell process has nothing to do and exits right away.
- **Confusing `stop` and `rm`** — `stop` halts a container but keeps it (and its writable layer / any un-persisted data) around; `rm` deletes the container permanently. A stopped container still counts against `docker ps -a` and still occupies disk space.
- **Port already in use** — `docker run -p 8080:80` fails if another process (including another container) already has host port 8080 bound; choose a different host port.
- **Name conflicts** — `docker run --name web ...` fails with “the container name is already in use” if a container (even a stopped one) already has that name; remove the old one first (`docker rm web`) or choose a new name.

Chapter 5: Environment Variables and Container Configuration

5.1 Why Environment Variables Matter

Containerized applications should be configurable **without rebuilding the image**. The most common way to achieve this is through environment variables, following the principles of the [Twelve-Factor App](#) methodology, which states that configuration (anything that varies between deploy environments — database URLs, credentials, feature flags) should live in the environment, not hard-coded in the application.

This lets the exact same image be promoted from development to staging to production, with only the environment variables changing between deployments.

5.2 Setting Environment Variables at Runtime

```
docker run -e APP_ENV=production myapp
docker run -e DB_HOST=db.example.com -e DB_PORT=5432 myapp
```

Multiple `-e` flags can be chained for multiple variables.

5.2.1 Loading from a File

For applications needing many variables, an env file is cleaner than a long list of `-e` flags:

```
# .env file
DB_HOST=db.example.com
DB_PORT=5432
DB_USER=admin
DB_PASSWORD=supersecret
```

```
docker run --env-file .env myapp
```

Security note: `.env` files often contain secrets. They should never be committed to version control (add them to `.gitignore`), and for production secrets, a dedicated secrets manager (Docker Secrets, HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets) is preferred over plain environment variables, since environment variables are visible via `docker inspect` and process listings.

5.3 Setting Default Environment Variables in a Dockerfile

Image authors can bake in sensible defaults using the `ENV` instruction:

```
FROM node:20
ENV NODE_ENV=production
ENV PORT=3000
```

Any value provided at `docker run` time with `-e` **overrides** the Dockerfile's `ENV` default. This layering — image-level defaults, overridden by runtime values — is the standard pattern for configurable containers.

5.4 Viewing a Container’s Environment Variables

```
docker exec CONTAINER env                # list all env vars inside a running
                                         # container

docker inspect CONTAINER \
  --format='{{range .Config.Env}}{{println .}}{{end}}'
```

5.5 Worked Example: A Configurable Database Container

The official MySQL image is a canonical example of environment-variable-driven configuration:

```
docker run -d \
  --name mydb \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  -e MYSQL_DATABASE=appdb \
  -e MYSQL_USER=appuser \
  -e MYSQL_PASSWORD=apppass \
  -p 3306:3306 \
  mysql:8
```

At startup, the MySQL image’s entrypoint script reads these variables and initializes the database accordingly — no manual configuration file editing required. This pattern (an entrypoint script that reads env vars and configures the application before handing off to the main process) is extremely common across official images (Postgres, MongoDB, Redis, RabbitMQ, and more).

5.6 Environment Variables in Docker Compose

Environment variables are equally central to Compose-based multi-container apps (fully covered in Chapter 7):

```
services:
  web:
    image: myapp
    environment:
      - APP_ENV=production
      - DB_HOST=db
    env_file:
      - .env
```

5.7 Best Practices

- Keep secrets out of Dockerfiles (ENV values become part of the image and are visible to anyone who can pull it or run `docker history`) — pass secrets at runtime instead.
- Use `.env` files for local development convenience, but use a real secrets manager in production.
- Document required environment variables (e.g., in a `README` or a `.env.example` template file) so consumers of the image know what to configure.
- Fail fast: applications should validate required environment variables at startup and exit with a clear error if something critical is missing, rather than failing mysteriously later.

Chapter 6: Docker Images and the Dockerfile

6.1 What Is an Image, Precisely?

A Docker image is an immutable, layered filesystem snapshot plus metadata describing how a container should run: the default command, exposed ports, working directory, environment variable defaults, and more. Images are identified by a **name** and a **tag** (e.g., `nginx:1.25`, `python:3.12-slim`), and internally by a content-addressable **digest** (a SHA-256 hash of the image manifest), which uniquely and immutably identifies the exact image content — two images with the same digest are guaranteed to be byte-for-byte identical, regardless of what tags point to them.

If no tag is specified, Docker defaults to the `latest` tag — which is simply a convention (often the most recently pushed image), **not** a guarantee of “the newest” or “the best” version. Relying on `latest` in production is discouraged, since the image it points to can change without warning; pinning to a specific version tag (or better, a digest) is best practice for reproducible deployments.

6.2 The Dockerfile

A **Dockerfile** is a plain-text file containing a sequence of instructions that Docker’s build engine executes in order to produce an image. Each instruction typically creates a new layer.

6.2.1 Core Instructions

```
FROM python:3.12-slim          # base image to build from - always first
                                # (except for ARG before FROM)

LABEL maintainer="team@example.com"  # arbitrary metadata

WORKDIR /app                   # sets the working directory for subsequent
                                # instructions and the container's default CWD

COPY requirements.txt .        # copy files from the build context into
                                # the image
ADD archive.tar.gz /app/       # like COPY, but also auto-extracts local
                                # tar archives and can fetch remote URLs
                                # (COPY is generally preferred for clarity)

RUN pip install --no-cache-dir -r requirements.txt  # execute a command
                                                    # at BUILD time, creating a new layer

COPY . .                       # copy the rest of the application code

ENV PORT=8000                  # set a default environment variable

EXPOSE 8000                    # documents which port the app listens on
                                # (does NOT actually publish the port -
                                # that's done with `docker run -p`)
```

```

USER appuser                                # switch from root to a non-root user for
                                           # the remaining instructions and runtime
                                           # (security best practice)

ENTRYPOINT ["python", "app.py"]            # the command that always runs when the
                                           # container starts (see 6.5 below)

CMD ["--port", "8000"]                     # default arguments to ENTRYPOINT (can be
                                           # overridden at `docker run` time)

```

6.2.2 Full Instruction Reference

Instruction	Purpose
FROM	Sets the base image
LABEL	Adds metadata as key-value pairs
RUN	Executes a command during the build, creating a layer
COPY	Copies files/directories from build context into the image
ADD	Like COPY, plus tar extraction and remote URL fetching
WORKDIR	Sets working directory for subsequent instructions
ENV	Sets a persistent environment variable
ARG	Defines a build-time-only variable (not present in the final container)
EXPOSE	Documents the port(s) the container listens on
VOLUME	Declares a mount point intended for persistent/external data
USER	Sets the user (and optionally group) for subsequent instructions and runtime
ENTRYPOINT	The fixed, primary command the container executes
CMD	Default arguments to ENTRYPOINT, or the default command if no ENTRYPOINT
HEALTHCHECK	Defines how Docker should test container health
STOPSIGNAL	Sets the system call signal used to stop the container
ONBUILD	Defers an instruction to run when this image is used as a base for another build

6.3 Building an Image

```
docker build -t myapp:1.0 .
```

- `-t myapp:1.0` — tags the resulting image with a name and version.

- `.` — the **build context**: the directory whose contents are sent to the Docker daemon and made available to `COPY/ADD` instructions. Everything in this directory (unless excluded — see 6.4) is sent, so keeping the build context lean matters for build speed.

Other useful build flags:

```
docker build -f Dockerfile.prod -t myapp:prod .      # use a non-default Dockerfile name/path
docker build --no-cache -t myapp:1.0 .              # rebuild ignoring layer cache entirely
docker build --build-arg VERSION=1.2.3 -t myapp .   # pass a build-time ARG value
docker build --progress=plain .                     # verbose, non-TTY build output
```

6.4 The `.dockerignore` File

Just as `.gitignore` excludes files from version control, `.dockerignore` excludes files from the build context sent to the daemon:

```
.git
node_modules
*.log
.env
Dockerfile
README.md
```

Excluding large, unnecessary directories (like `node_modules` or `.git`) both speeds up builds and avoids accidentally baking secrets or bloat into the image.

6.5 Layer Caching in Depth

Docker’s build cache works instruction-by-instruction, top to bottom: as soon as one instruction’s cache is invalidated (its inputs changed), **every subsequent instruction is rebuilt from scratch**, even if their own inputs didn’t change. This means **instruction order matters enormously** for build speed.

6.5.1 Anti-pattern (slow rebuilds)

```
FROM node:20
WORKDIR /app
COPY . .                # copies EVERYTHING, including source code
RUN npm install          # this reruns on every single code change,
                        # even a one-line change to app.js, because
                        # COPY . . invalidated the cache above it

CMD ["node", "server.js"]
```

6.5.2 Best practice (fast rebuilds)

```
FROM node:20
WORKDIR /app
COPY package*.json ./   # copy ONLY dependency manifests first
RUN npm install          # this layer is cached and skipped unless
                        # package.json/package-lock.json change
```

```

COPY . . # copy the rest of the source code - this
# invalidates cache only for THIS and later
# layers, not the (expensive) npm install

CMD ["node", "server.js"]

```

This reordering — copying dependency manifests and installing dependencies *before* copying application source code — is one of the single highest-leverage Dockerfile optimizations, since dependency installation is typically far slower than copying a few source files, and source code changes far more often than dependencies.

6.6 Multi-Stage Builds

Multi-stage builds let a single Dockerfile use multiple `FROM` instructions, where later stages can selectively copy artifacts from earlier stages — without carrying forward the earlier stage’s build tools, source code, or intermediate files into the final image.

```

# ---- Stage 1: build ----
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # produces static files in /app/dist

# ---- Stage 2: production ----
FROM nginx:1.25-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Benefits:

- **Smaller final images** — the final image contains only Nginx and the compiled static files, not Node.js, npm, `node_modules`, or source code, potentially shrinking an image from over a gigabyte down to tens of megabytes.
- **Improved security** — fewer tools and less code in the production image means a smaller attack surface.
- **Simplified pipelines** — no need for separate build scripts outside Docker; the entire build-and-package process is captured in one Dockerfile.

Multi-stage builds are the standard, recommended approach for compiled languages (Go, Java, Rust, C++) and for frontend build tooling (React, Vue, Angular) alike.

6.7 Choosing a Base Image

Base image style	Example	Size	Trade-off
Full OS	ubuntu:22.04	~70MB+	Familiar tooling, larger

Base image style	Example	Size	Trade-off
Slim variant	<code>python:3.12-slim</code>	Smaller	Fewer preinstalled packages
Alpine-based	<code>node:20-alpine</code>	Very small (~5-10MB base)	Uses musl libc instead of glibc, which occasionally causes subtle compatibility issues with compiled dependencies
Distroless	<code>gcr.io/distroless/*</code>	Minimal	No shell, no package manager — maximal security, harder to debug
<code>scratch</code>	<code>FROM scratch</code>	~0MB	Truly empty; only for statically compiled binaries (common with Go)

6.8 CMD vs ENTRYPOINT

This distinction is one of the most frequently misunderstood parts of Docker, and worth covering in depth.

6.8.1 The Two Forms

Both CMD and ENTRYPOINT support two syntaxes:

- **Exec form (preferred):** `CMD ["executable", "arg1", "arg2"]` — runs the command directly, without a shell. Signals (like `SIGTERM` from `docker stop`) are received directly by the process (PID 1), enabling graceful shutdown.
- **Shell form:** `CMD command arg1 arg2` — Docker wraps the command in `/bin/sh -c "..."`, meaning the shell (not your application) becomes PID 1, which can interfere with proper signal handling.

6.8.2 CMD Alone

CMD specifies the default command to run when a container starts, **if no command is given at `docker run` time**. It can be fully overridden.

```
FROM ubuntu
CMD ["echo", "Hello, World"]
```

```
docker run myimage           # prints "Hello, World"
docker run myimage echo "Hi" # CMD is entirely replaced: prints "Hi"
```

6.8.3 ENTRYPOINT Alone

ENTRYPOINT specifies a command that **always** runs; any command-line arguments given at `docker run` are **appended** to it, not used to replace it.

```
FROM ubuntu
ENTRYPOINT ["echo"]
```

```
docker run myimage "Hello"          # runs: echo Hello
docker run myimage "Hi there"       # runs: echo Hi there
```

To override the ENTRYPOINT itself (not just append arguments), you must explicitly pass `--entrypoint`:

```
docker run --entrypoint ls myimage -la /
```

6.8.4 Combining ENTRYPOINT and CMD (the recommended pattern)

The most powerful and common pattern combines both: ENTRYPOINT defines the fixed executable, and CMD supplies **default arguments** to it, which the user can still override.

```
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["Hello, World"]
```

```
docker run myimage          # runs: echo Hello, World    (uses CMD default)
docker run myimage "Custom Text" # runs: echo Custom Text    (overrides CMD)
```

This pattern is exactly how many official images work — for example, the official `postgres` image uses `ENTRYPOINT ["docker-entrypoint.sh"]` with `CMD ["postgres"]`, so the entrypoint script always runs (handling initialization), while users can still override what command it ultimately execs into.

6.8.5 Summary Table

	Can be overridden by <code>docker run ARGS</code> ?	Typical use
CMD only	Fully replaced	Simple default command
ENTRYPOINT only	Arguments appended, executable fixed	Force a specific executable always
ENTRYPOINT + CMD	CMD provides overridable default <i>arguments</i>	Most flexible; recommended for images meant to behave like a CLI tool

6.9 HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1
```

Docker periodically runs the check command inside the container; `docker ps` then reports the container's health as **healthy**, **unhealthy**, or **starting**. This is especially valuable in orchestrated environments (Swarm, Kubernetes with similar liveness/readiness probes), which use health status to decide whether to route traffic to a container or restart it.

6.10 Image Naming, Tagging, and Versioning

```
docker tag myapp:1.0 myrepo/myapp:1.0
docker tag myapp:1.0 myrepo/myapp:latest
```

A full image reference has the form:

[REGISTRY_HOST[:PORT]/]REPOSITORY[:TAG]

- If `REGISTRY_HOST` is omitted, Docker Hub is assumed.
- Common versioning conventions: semantic version tags (`1.4.2`), a `latest` tag for the most recent stable build, and `git` short-SHA tags (`a1b2c3d`) for precise, immutable traceability from image back to source commit — a very common CI/CD pattern.

Chapter 7: Docker Networking

7.1 Why Networking Matters

Real applications are rarely a single container — a web server needs to talk to a database, a cache, and perhaps other microservices, and needs to be reachable from outside the host. Docker’s networking subsystem controls how containers communicate with each other, with the host, and with the outside world.

7.2 Network Drivers

Docker ships with several built-in network drivers, each suited to a different scenario:

Driver	Scope	Typical Use
bridge	Single host	Default; isolated private network for containers on one host
host	Single host	Container shares the host’s network stack directly — no isolation, no port mapping needed
none	Single host	No networking at all — total isolation
overlay	Multi-host (Swarm)	Connects containers across multiple Docker hosts (Chapter 11)
macvlan	Single host	Assigns a container a MAC address, making it appear as a physical device on the network
ipvlan	Single host	Similar to macvlan but shares a single MAC address

7.3 The Default Bridge Network

When Docker is installed, it automatically creates a default bridge network named **bridge**. Any container run without an explicit **--network** flag attaches to it.

```
docker network ls          # list all networks
docker network inspect bridge # view details: subnet, gateway, connected
                             # containers
```

Characteristics of the **default** bridge network:

- Containers get an IP address on an internal, private subnet (commonly 172.17.0.0/16).
- Containers can reach each other **by IP address**, but — importantly — **not by container name** (no built-in DNS resolution on the default bridge network). This is a common source of confusion for beginners.
- Containers can reach the outside internet (via NAT through the host), and can be reached from outside via explicit port mapping (**-p**).

7.4 User-Defined Bridge Networks (Recommended)

Because the default bridge network lacks automatic DNS-based service discovery, Docker strongly recommends creating custom (user-defined) bridge networks for any multi-container application:

```
docker network create mynetwork
docker run -d --name db --network mynetwork postgres
docker run -d --name web --network mynetwork myapp
```

On a **user-defined** bridge network:

- Docker runs an embedded DNS server, so containers can resolve each other **by container name** automatically (**web** can connect to **db** simply by hostname **db** — no IP addresses needed).
- Only containers explicitly attached to the same user-defined network can communicate with each other by default — improving isolation between unrelated application stacks on the same host.
- Containers can be connected to (and disconnected from) networks dynamically, even after they're running: `docker network connect NETWORK CONTAINER`.

This name-based service discovery is foundational to how Docker Compose wires multi-container applications together (Chapter 7 of Compose usage / Chapter 8 overall) — Compose automatically creates a user-defined network for each project.

7.5 The host Network

```
docker run -d --network host nginx
```

The container shares the host machine's network namespace directly — there is no isolation and no need for `-p` port publishing, since the container's ports *are* the host's ports. This offers the best possible network performance (no NAT overhead) but sacrifices isolation and can cause port conflicts with services already running on the host. It is Linux-only; it behaves differently (or is unsupported) on Docker Desktop for Mac/Windows because of the underlying VM.

7.6 The none Network

```
docker run -d --network none myapp
```

The container gets a network namespace with only a loopback interface — no external connectivity at all. Useful for batch-processing containers that only need local disk/CPU, or for maximal network isolation.

7.7 Publishing Ports

```
docker run -p HOST_PORT:CONTAINER_PORT IMAGE
docker run -p 8080:80 nginx           # host:8080 → container:80
docker run -p 127.0.0.1:8080:80 nginx # bind only to localhost, not all
                                       # host interfaces
docker run -p 8080:80 -p 8443:443 nginx # publish multiple ports
```

```
docker run -P nginx # publish ALL EXPOSEd ports to
# random, ephemeral host ports
```

Under the hood, on Linux, port publishing works via **iptables NAT rules** that Docker manages automatically, forwarding traffic arriving on the host port into the container's network namespace.

7.8 Inspecting Network Configuration

```
docker network ls
docker network inspect NETWORK
docker network create --driver bridge --subnet 172.20.0.0/16 mynet
docker network connect NETWORK CONTAINER # attach a running container
# to an additional network

docker network disconnect NETWORK CONTAINER
docker network rm NETWORK
docker network prune # remove unused networks
```

Finding a specific container's IP address:

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' CONTAINER
```

7.9 Container-to-Container Communication: Worked Example

```
# 1. Create a dedicated network
docker network create appnet

# 2. Start a database, attached to that network
docker run -d --name db --network appnet \
  -e POSTGRES_PASSWORD=secret postgres:16

# 3. Start an app that connects to the database BY NAME
docker run -d --name api --network appnet \
  -e DATABASE_URL=postgres://postgres:secret@db:5432/postgres \
  myapi

# The `api` container can resolve the hostname "db" automatically,
# thanks to the embedded DNS server on the user-defined network.
```

7.10 DNS and Service Discovery Summary

Network type	Container-name DNS resolution?
Default bridge	No
User-defined bridge	Yes
host	N/A (shares host's network stack and DNS)
Swarm overlay	Yes, plus built-in load balancing across replicas

7.11 Common Pitfalls

- Expecting containers on the **default** bridge network to resolve each other by name — they can't; either use a user-defined network or fall back to `--link` (legacy, discouraged) or IP addresses.
- Forgetting that **EXPOSE** in a Dockerfile is purely documentation — it does **not** publish a port. Only `-p/-P` at `docker run` time (or the `ports:` section in Compose) actually opens a port to the host.
- Binding to `0.0.0.0` inside the application — if an app inside a container listens only on `127.0.0.1`, it will not be reachable from outside the container even with correct port publishing, since the loopback interface is container-local.
- Port conflicts on the host when running multiple containers that all try to map to the same host port.

Chapter 8: Docker Storage — Volumes, Bind Mounts, and tmpfs

8.1 Why Persistent Storage Is Necessary

Recall from Chapter 2 that a container’s writable layer is deleted permanently when the container is removed. For stateless applications, this is fine — but for anything that needs to retain data across container restarts or removals (databases, uploaded files, logs meant to survive redeployments), Docker provides three storage mechanisms, each with different trade-offs.

8.2 The Three Storage Mechanisms

Mechanism	Managed by	Location	Typical use
Volumes	Docker	Docker-managed area on host (<code>/var/lib/docker/volumes</code> on Linux)	Recommended default for persisting application data
Bind mounts	You (the user)	Any path on the host filesystem	Mounting source code for local development; sharing specific host files/dirs
tmpfs mounts	Docker (in-memory)	RAM only, never written to disk	Sensitive, temporary data (e.g., secrets held briefly in memory); Linux only

```
+-----+
|                                     |
|               Host Machine         |
|                                     |
|  /var/lib/docker/volumes/myvol/_data  <-- Volume      |
|                                     | (Docker-managed) |
|  /home/user/myproject/src             <-- Bind mount  |
|                                     | (arbitrary path) |
|                                     | RAM <-- tmpfs mount (never touches disk) |
+-----+
```

8.3 Volumes

Volumes are the Docker-recommended mechanism for persisting data because they are fully managed by Docker (making them portable, easy to back up, and independent of the host’s directory structure), and they work identically across Linux, Mac, and Windows.

8.3.1 Creating and Using Volumes

```
docker volume create mydata          # explicitly create a named volume
docker volume ls                     # list all volumes
docker volume inspect mydata         # view details: mountpoint, driver
docker volume rm mydata              # remove a volume (must be unused)
docker volume prune                  # remove all unused volumes
```

```
docker run -d --name db \
-v mydata:/var/lib/postgresql/data \
postgres:16
```

Here, `mydata` is a named volume mounted at `/var/lib/postgresql/data` inside the container. If `mydata` doesn't already exist, Docker creates it automatically on first use. Even if the `db` container is removed, the volume — and the data inside it — persists until explicitly deleted.

8.3.2 Anonymous Volumes

```
docker run -d -v /var/lib/postgresql/data postgres:16
```

Omitting a name creates an **anonymous** volume with a random hash-like name. These are easy to lose track of (and easy to accidentally accumulate via `docker volume prune` cleanup), so named volumes are generally preferred for anything meant to persist meaningfully.

8.3.3 Sharing a Volume Across Multiple Containers

```
docker volume create shared_data
docker run -d --name writer -v shared_data:/data busybox sh -c "while true; do date >> /data/log.txt"
docker run -d --name reader -v shared_data:/data:ro busybox sh -c "tail -f /data/log.txt"
```

The `:ro` suffix mounts the volume **read-only** in the `reader` container, while `writer` retains read-write access — a common pattern for enforcing least-privilege access to shared data.

8.4 Bind Mounts

Bind mounts map an exact path on the host filesystem directly into the container. Unlike volumes, the host path is fully under the user's control and doesn't need to live in Docker's managed storage area.

```
docker run -d --name web \
-v /home/user/site:/usr/share/nginx/html:ro \
-p 8080:80 \
nginx
```

8.4.1 The Modern `--mount` Syntax

While `-v/--volume` is the traditional shorthand, the more explicit and verbose `--mount` flag is recommended in scripts for clarity, as it makes the mount type explicit and rejects invalid combinations that `-v` would silently misinterpret:

```
docker run -d --name web \
--mount type=bind,source=/home/user/site,target=/usr/share/nginx/html,readonly \
-p 8080:80 \
nginx

docker run -d --name db \
```

```
--mount type=volume,source=mydata,target=/var/lib/postgresql/data \
postgres:16
```

8.4.2 Bind Mounts for Local Development

A hugely common development pattern is bind-mounting source code into a container so that code edited on the host is immediately reflected inside the running container, without rebuilding the image on every change:

```
docker run -it --rm \
  -v $(pwd):/app \
  -w /app \
  -p 3000:3000 \
  node:20 npm run dev
```

Combined with a framework’s hot-reload/file-watcher capability, this gives a fast local development loop while still running inside a container that matches production.

8.5 tmpfs Mounts (Linux Only)

```
docker run -d --tmpfs /app/cache:rw,size=64m myapp
```

tmpfs mounts exist purely in host memory (RAM) and are never written to disk — data vanishes when the container stops. Useful for temporary, sensitive data (like decrypted secrets used briefly during startup) that should never be persisted to disk at all.

8.6 Volume Drivers and Cloud Storage

Docker’s volume subsystem is pluggable via **volume drivers**, enabling volumes backed by network or cloud storage rather than the local disk — for example, drivers exist for NFS, Amazon EBS/EFS, Azure Files, and various distributed storage systems. This is particularly relevant in orchestrated, multi-host environments (Swarm, Kubernetes), where a container might be scheduled onto any node in the cluster, and local-disk volumes wouldn’t “follow” it.

```
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw \
  --opt device=/path/to/share \
  nfs_volume
```

8.7 Backing Up and Restoring a Volume

A common pattern uses a temporary helper container to tar up a volume’s contents:

```
# Backup
docker run --rm -v mydata:/data -v $(pwd):/backup \
  busybox tar czf /backup/mydata_backup.tar.gz -C /data .

# Restore into a fresh volume
docker volume create mydata_restored
```

```
docker run --rm -v mydata_restored:/data -v $(pwd):/backup \
  busybox tar xzf /backup/mydata_backup.tar.gz -C /data
```

8.8 Choosing Between Volumes and Bind Mounts

Question	Answer favors
Does data need to survive container removal?	Volume
Do you need to edit files live from your host during development?	Bind mount
Should Docker manage the storage location for you?	Volume
Do you need to share a specific existing host directory (e.g., config files)?	Bind mount
Will this run across multiple hosts in a cluster?	Volume (with a suitable network-backed driver)
Is the data sensitive and must never touch disk?	tmpfs

Chapter 9: Docker Compose — Multi-Container Applications

9.1 The Problem Compose Solves

Real applications are rarely a single container. A typical web application might need: a web/API server, a database, a cache (Redis), and perhaps a reverse proxy — each in its own container, all networked together, with specific startup order, volumes, and environment configuration.

Managing this by hand with a sequence of `docker network create`, `docker volume create`, and multiple `docker run` commands (each with many flags) is tedious, error-prone, and hard to version-control or share with teammates. **Docker Compose** solves this by letting you describe an entire multi-container application declaratively in a single YAML file, and bring the whole stack up or down with one command.

9.2 Installing Compose

Modern Docker installations include Compose as a plugin, invoked as `docker compose` (with a space) — the successor to the older, separately-installed Python tool `docker-compose` (with a hyphen), which is now in maintenance mode. Both share nearly identical syntax; new setups should use `docker compose`.

```
docker compose version
```

9.3 Anatomy of a `docker-compose.yml` File

```
version: "3.9"           # Compose file format version (often omitted in
                        # modern Compose, which auto-detects)

services:                # each service becomes one container (or more,
                        # if scaled)

  web:
    build: ./web          # build from a local Dockerfile
    ports:
      - "8080:80"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
    depends_on:
      - db
    networks:
      - appnet
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=appdb
    volumes:
```

```

    - dbdata:/var/lib/postgresql/data
networks:
    - appnet
restart: unless-stopped

cache:
    image: redis:7-alpine
    networks:
        - appnet

volumes:                                # top-level volume declarations
    dbdata:

networks:                                # top-level network declarations
    appnet:
        driver: bridge

```

9.3.1 Key Sections Explained

Key	Purpose
services	One entry per container/service in the application
build	Build an image from a local Dockerfile (path to build context)
image	Use a pre-built image instead of building
ports	Host:container port mappings, equivalent to <code>-p</code>
environment	Environment variables, equivalent to <code>-e</code>
env_file	Load environment variables from a file
volumes (service-level)	Mount volumes/bind mounts into the service
volumes (top-level)	Declares named volumes usable by any service
networks	Attach services to specific networks
depends_on	Controls startup order (not full readiness — see 9.7)
restart	Restart policy, same values as <code>docker run --restart</code>
command	Override the image's default command
entrypoint	Override the image's entrypoint

9.4 Core Compose Commands

```

docker compose up                                # create and start all services
docker compose up -d                             # detached (background) mode
docker compose up --build                         # force rebuild of images before starting
docker compose down                              # stop and remove containers, networks
                                                # (but NOT named volumes, by default)
docker compose down --volumes                    # also remove named volumes - deletes data!

```

```

docker compose ps           # list containers in this project
docker compose logs        # view logs for all services
docker compose logs -f SERVICE # follow logs for one service
docker compose exec SERVICE bash # shell into a running service
docker compose build        # build (or rebuild) images without
                             # starting containers
docker compose stop        # stop containers without removing them
docker compose start        # start previously-stopped containers
docker compose restart SERVICE # restart a single service
docker compose config       # validate and print the fully
                             # resolved configuration

```

9.5 Scaling Services

```
docker compose up -d --scale web=3
```

This starts three replicas of the `web` service. Note that if `web` has a fixed host port mapping (e.g., `8080:80`), scaling will fail with a port conflict, since multiple containers can't bind the same host port — scaled services typically omit fixed host ports and rely on a load balancer/reverse proxy in front, or let Docker assign ephemeral ports.

9.6 Multiple Compose Files and Overrides

Compose supports layering multiple files, which is the standard way to handle differences between development and production configuration:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

A typical pattern:

- `docker-compose.yml` — base configuration common to all environments.
- `docker-compose.override.yml` — automatically merged in by Compose during local development (e.g., bind-mounting source code, enabling debug ports).
- `docker-compose.prod.yml` — production overrides (e.g., stricter resource limits, no bind mounts, different `restart` policy).

```

# docker-compose.override.yml (auto-loaded in dev)
services:
  web:
    volumes:
      - ./web:/app
    command: npm run dev

```

9.7 depends_on and Startup Ordering

```

services:
  web:
    depends_on:
      db:

```

```

        condition: service_healthy
db:
  image: postgres:16
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 3s
    retries: 5

```

A common misunderstanding: plain `depends_on` only guarantees **start order**, not that the dependency is actually *ready* to accept connections (a database container can be “started” long before its database engine finishes initializing). The `condition: service_healthy` form (requiring a `healthcheck` on the dependency) properly waits for real readiness. Alternatively, applications themselves should implement retry/backoff logic when connecting to dependencies — the more robust, orchestration-agnostic solution, since this same “not actually ready yet” problem exists identically in Kubernetes and other orchestrators.

9.8 Environment Variable Substitution

Compose files can reference host environment variables or a `.env` file located alongside the Compose file:

```

services:
  web:
    image: "myapp:${TAG:-latest}"      # defaults to "latest" if TAG is unset
    ports:
      - "${WEB_PORT}:80"

```

```

# .env
TAG=1.4.2
WEB_PORT=8080

```

9.9 Worked Example: A Full-Stack App (Frontend, API, Database, Cache)

```

services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - api

  api:
    build: ./api
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgres://postgres:secret@db:5432/appdb
      - REDIS_URL=redis://cache:6379

```

```

depends_on:
  db:
    condition: service_healthy
  cache:
    condition: service_started

db:
  image: postgres:16
  environment:
    - POSTGRES_PASSWORD=secret
    - POSTGRES_DB=appdb
  volumes:
    - dbdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    retries: 5

cache:
  image: redis:7-alpine

volumes:
  dbdata:

```

Bringing up this entire four-service application, correctly networked with name-based service discovery (db, cache, api are all resolvable by name — see Chapter 7), is a single command: `docker compose up -d`.

9.10 Compose vs. Swarm vs. Kubernetes

It's worth clarifying scope early, before Chapters 11–12 cover orchestration in depth: **Compose is designed for single-host** deployments — local development, testing, small single-server production setups, and CI pipelines. It does not handle multi-host clustering, automatic failover across machines, or rolling updates across a fleet. For that, Docker Swarm (which can actually consume a Compose file directly via `docker stack deploy`) or Kubernetes are used — covered in Chapters 11 and 12.

Chapter 9b: Advanced Docker Compose Features

Building on the Compose fundamentals from Chapter 9, this section covers features that become useful once an application grows beyond a handful of simple services.

9b.1 Profiles — Selectively Enabling Services

Profiles let a single Compose file describe services that are only started in certain contexts (e.g., debugging tools, optional services, seed/migration jobs), without needing separate Compose files for each variation.

```
services:
  web:
    image: myapp
    # no profiles: field means this service always starts

  adminer:
    image: adminer
    ports: ["8081:8080"]
    profiles: ["debug"]

  migrate:
    image: myapp-migrator
    profiles: ["setup"]
```

```
docker compose up -d                # only "web" starts
docker compose --profile debug up -d # "web" and "adminer" start
docker compose --profile setup run migrate
```

9b.2 extends and YAML Anchors for Reuse

For services sharing significant configuration, YAML anchors avoid repetition within a single file:

```
x-common-env: &common-env
  LOG_LEVEL: info
  TZ: UTC

services:
  api:
    image: myapi
    environment:
      <<: *common-env
      SERVICE_NAME: api

  worker:
    image: myworker
    environment:
      <<: *common-env
      SERVICE_NAME: worker
```

9b.3 Resource Limits in Compose

Even outside of Swarm's `deploy:` key (Chapter 14), plain `docker compose up` supports resource constraints directly:

```
services:
  api:
    image: myapi
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          memory: 256M
```

Note: under plain `docker compose up` (not `docker stack deploy`), only `resources.limits` are actually enforced; the full `deploy:` block (replicas, `restart_policy`, placement) is primarily meaningful in Swarm mode.

9b.4 `docker compose watch` — Live Development Sync

Newer Compose versions include a built-in file-watching mode, reducing the need for manual bind-mount-plus-dev-server setups described in Chapter 9:

```
services:
  api:
    build: ./api
    develop:
      watch:
        - action: sync
          path: ./api/src
          target: /app/src
        - action: rebuild
          path: ./api/package.json
```

```
docker compose watch
```

Changes to `./api/src` are synced live into the running container; changes to `package.json` trigger a full image rebuild — giving fine-grained control over which changes need a full rebuild versus a fast in-place sync.

9b.5 Dependency Conditions Beyond `service_healthy`

```
services:
  migrate:
    image: myapp-migrator
    depends_on:
      db:
        condition: service_healthy
```

```
api:
  image: myapi
  depends_on:
    migrate:
      condition: service_completed_successfully
```

`condition: service_completed_successfully` is especially useful for one-shot setup/migration jobs that must finish successfully before dependent long-running services start — a common pattern for database schema migrations.

9b.6 Named Compose Projects

By default, Compose derives a project name from the current directory, prefixing all created resources (containers, networks, volumes) with it. Running the same Compose file from different directories (or explicitly naming projects) allows multiple independent copies of the same application stack to run side by side on one host — handy for running several feature-branch environments locally at once:

```
docker compose -p myapp-feature-x up -d
docker compose -p myapp-feature-y up -d
```

Chapter 10: Docker Registries and Image Distribution

10.1 What Is a Registry?

A **registry** is a storage and content-delivery system for container images. A registry organizes images into **repositories** (a named collection of related images, distinguished by tags), and repositories can further be organized into **namespaces**, typically corresponding to a user or organization account.

```
docker.io / library / nginx : 1.25
|         |         |         |
registry namespace repo      tag
```

When you run `docker pull nginx`, Docker expands this shorthand to the full reference `docker.io/library/nginx:latest` — `docker.io` (Docker Hub) is the default registry, and `library` is the namespace reserved for official, Docker-curated images.

10.2 Docker Hub

Docker Hub (`hub.docker.com`) is the default, most widely used public registry:

- **Official Images** — curated, actively maintained images for popular software (nginx, postgres, node, python, redis, etc.), vetted by Docker and upstream maintainers for security and best practices.
- **Verified Publisher / Sponsored OSS images** — images published by known organizations.
- **Public user/organization repositories** — anyone can push public images for free.
- **Private repositories** — a limited number free, more with a paid plan, for proprietary images.

10.2.1 Authenticating

```
docker login                                # prompts for Docker Hub username/password
                                           # (or personal access token, recommended
                                           # over passwords)
docker login myregistry.example.com        # login to a different/private registry
docker logout
```

10.3 Pushing and Pulling Images

```
docker pull nginx:1.25                    # pull explicitly
docker pull myregistry.example.com/team/app:1.0 # pull from a private registry

# To push, an image must first be TAGGED with the target repository name
docker tag myapp:1.0 myusername/myapp:1.0
docker push myusername/myapp:1.0

# Private registry example
docker tag myapp:1.0 myregistry.example.com/team/myapp:1.0
docker push myregistry.example.com/team/myapp:1.0
```

Because registries are content-addressable, pushing/pulling is efficient: only layers not already present on the destination are actually transferred, thanks to the shared-layer model described in Chapter 2.

10.4 Running a Private Registry

For organizations that can't (or don't want to) rely on a third-party public registry, Docker provides an official open-source registry image that can be self-hosted:

```
docker run -d -p 5000:5000 --name registry \
  -v registry_data:/var/lib/registry \
  registry:2
```

Tag and push to the local private registry

```
docker tag myapp:1.0 localhost:5000/myapp:1.0
```

```
docker push localhost:5000/myapp:1.0
```

```
docker pull localhost:5000/myapp:1.0
```

Production private registries are typically secured with TLS and authentication, and are commonly provided as managed services rather than self-hosted from scratch:

Provider	Service
AWS	Elastic Container Registry (ECR)
Google Cloud	Artifact Registry / Container Registry
Microsoft Azure	Azure Container Registry (ACR)
GitHub	GitHub Container Registry (GHCR)
GitLab	GitLab Container Registry
Self-hosted	Harbor (adds vulnerability scanning, RBAC, replication on top of the base registry)

10.5 Image Naming Conventions for Registries

[REGISTRY_HOST[:PORT]/]NAMESPACE/REPOSITORY[:TAG]

Examples:

nginx:1.25	→ Docker Hub, official image
myusername/myapp:1.0	→ Docker Hub, personal namespace
ghcr.io/myorg/myapp:1.0	→ GitHub Container Registry
123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0	→ AWS ECR

10.6 Image Digests and Immutability

Tags are mutable — `myapp:latest` can point to different actual content over time as new versions are pushed. For fully reproducible deployments, images can instead be referenced by their immutable **digest**:

```
docker pull myapp@sha256:3f8a4d9e1b2c...
```

CI/CD pipelines commonly resolve a tag to its digest at build time and deploy using the digest, guaranteeing that exactly the tested artifact — and nothing that might have been pushed to the same tag afterward — is what actually reaches production.

10.7 Vulnerability Scanning

Most registries (Docker Hub, ECR, GHCR, and self-hosted Harbor) offer automated **image vulnerability scanning**, comparing installed package versions inside an image against known-CVE databases. Integrating scanning into the CI pipeline — failing a build if critical vulnerabilities are found — is a standard DevSecOps practice, covered further in Chapter 14.

```
docker scout cveS myapp:1.0           # Docker's built-in scanning tool (Docker Scout)
```

10.8 Rate Limits and Caching

Docker Hub enforces pull rate limits for anonymous and free-tier authenticated users, which has led many CI systems and production clusters to run a **pull-through cache/mirror** in front of Docker Hub, reducing redundant external pulls and avoiding rate-limit failures during busy build periods.

Chapter 11: The Docker Engine, Deep Dive

11.1 Recap and Expansion

Chapter 2 introduced the client-server architecture and the containerd/runc chain. This chapter goes deeper into how the Engine actually isolates and runs containers, and how it's configured and monitored operationally.

11.2 The Full Component Stack

```
docker CLI -- talks to the REST API
|
dockerd (Docker daemon)
| - image management, build (via BuildKit), networking, volumes
| - exposes REST API
| - delegates container execution to containerd
v
containerd
| - image pull/push, storage/snapshotting, container lifecycle
| - exposes a gRPC API, used directly by Kubernetes' CRI shim too
v
containerd-shim-runc-v2 (one per running container)
| - decouples container process lifetime from containerd's own
| - reports exit status, keeps STDIO streams open
v
runc
| - OCI-compliant; does the actual `clone()` with namespace flags,
|   sets up cgroups, pivots root filesystem, then execs the
|   container's process
v
The container's process (running with PID 1 inside its own PID namespace)
```

11.3 BuildKit

Modern Docker builds (`docker build` / `docker buildx build`) use **BuildKit**, a from-scratch redesign of the image-building engine (default since Docker 23.0), offering:

- **Parallel execution** of independent build stages (relevant to multi-stage builds, Chapter 6)
- **Better caching**, including cache export/import to/from a registry — enabling build caches to be shared across CI runners, not just reused on a single machine
- **Build secrets** (`--secret`) — mount a secret file into a build stage without it ever being persisted into an image layer, unlike `ARG/ENV`, which do end up baked into the image history
- **buildx** — a CLI plugin providing multi-platform builds (e.g., building `linux/amd64` and `linux/arm64` images from a single invocation, using QEMU emulation or native multi-arch builders)

```
docker buildx build --platform linux/amd64,linux/arm64 -t myapp:1.0 --push .
```

```
docker build --secret id=npmr,src=$HOME/.npmrc -t myapp .
```

```
# in the Dockerfile:
# RUN --mount=type=secret,id=npnrc cp /run/secrets/npmrc ~/.npmrc && npm install
```

11.4 Storage Drivers

The Docker daemon uses a **storage driver** to implement the layered, copy-on-write filesystem described in Chapter 2. The modern default and recommended driver on Linux is **overlay2**, built on the kernel's OverlayFS. Other drivers (**aufs**, **devicemapper**, **btrfs**, **zfs**) exist for specific legacy or specialized use cases but are rarely the right default choice today.

```
docker info | grep "Storage Driver"
```

11.5 The Container Runtime Spec (OCI)

The **Open Container Initiative** defines two key specifications that keep the ecosystem interoperable:

- **Image Specification** — the standard format for container images (manifest, layers, config), which is why an image built by Docker can be run by **podman**, **containerd**, or a Kubernetes node without modification.
- **Runtime Specification** — the standard contract for “how to run a container” (an on-disk bundle plus a **config.json** describing namespaces, mounts, and process to run), which **runc** implements, and which alternative low-level runtimes (like **crun**, or the sandboxed **gVisor**/**kata-containers** runtimes used for stronger isolation) also implement, making them drop-in-compatible replacements for **runc**.

11.6 Daemon Configuration

The daemon reads configuration from **/etc/docker/daemon.json** on Linux (or via Docker Desktop's GUI settings on Mac/Windows):

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "default-address-pools": [
    { "base": "172.30.0.0/16", "size": 24 }
  ],
  "insecure-registries": ["myregistry.local:5000"],
  "registry-mirrors": ["https://mirror.gcr.io"]
}
```

After editing, restart the daemon: `sudo systemctl restart docker`.

11.7 Logging Drivers

By default, Docker captures container **STDOUT**/**STDERR** using the **json-file** log driver, which is what **docker logs** reads from. For production, container logs are commonly shipped to a centralized system instead, via alternative log drivers:

Driver	Destination
<code>json-file</code> (default)	Local JSON files on the host
<code>journald</code>	systemd's journal
<code>syslog</code>	A syslog server
<code>fluentd</code>	Fluentd, commonly forwarding into Elasticsearch/OpenSearch
<code>awslogs</code>	AWS CloudWatch Logs
<code>gelf</code>	Graylog Extended Log Format endpoint (e.g., Graylog, Logstash)

```
docker run -d --log-driver=journald myapp
```

Without a `max-size`/`max-file` limit on the default `json-file` driver, container logs can grow unbounded and fill host disk — a common, painful production incident. Always set log rotation limits.

11.8 Resource Isolation and Limits Revisited

Recall from Chapter 1 that cgroups implement resource limiting. Concretely, at `docker run` time:

```
docker run -d --memory=512m --memory-swap=1g --cpus="1.5" \
  --cpu-shares=512 --pids-limit=100 myapp
```

- `--memory` — hard memory cap; the kernel's OOM killer terminates the container's processes if exceeded.
- `--cpus` — caps CPU usage (e.g., 1.5 cores max, even on a multi-core host).
- `--cpu-shares` — a *relative* weight used only when there's CPU contention between containers (not a hard cap).
- `--pids-limit` — caps the number of processes/threads a container can spawn, guarding against fork bombs.

11.9 Security Features of the Engine

- **Namespaces & cgroups** (Chapter 1) — the foundational isolation primitives.
- **Linux capabilities** — by default, Docker drops many Linux root capabilities even for containers running as root, rather than granting full root privileges. `--cap-add`/`--cap-drop` fine-tune this.
- **Seccomp profiles** — restrict which system calls a container's processes may invoke; Docker applies a sensible default seccomp profile automatically.
- **AppArmor / SELinux** — mandatory access control systems that can further confine container processes on supporting Linux distributions.
- **Rootless mode** — an alternative daemon mode where `dockerd` itself runs as a non-root user, reducing the blast radius if the daemon is compromised.
- **User namespaces** — can remap container root (UID 0) to an unprivileged UID on the host, so that “root in the container” is not actually root on the host.

Chapter 11b: Beyond Docker — The Broader Container Ecosystem

Docker popularized containers, but the ecosystem that grew around the OCI standards it helped establish (Chapter 11) now includes several alternative and complementary tools worth knowing about.

11b.1 Podman

Podman is a daemonless container engine, developed primarily by Red Hat, designed as a largely drop-in-compatible alternative to the **docker** CLI (many commands work identically — **podman run**, **podman build**, **podman ps**).

Key differences from Docker:

- **Daemonless architecture** — Podman doesn’t run a persistent background daemon; each **podman** command runs and manages containers directly, which some consider a simpler and more secure model (no long-running root-privileged process to compromise).
- **Rootless by default** — Podman was built from the ground up with rootless operation as a first-class use case, letting unprivileged users run containers without any special group membership (contrast with Docker’s **docker** group, Chapter 3).
- **Pod support natively** — Podman can group containers into “pods” directly at the CLI level, mirroring Kubernetes’ Pod concept (Chapter 15), and can even generate Kubernetes YAML from a running set of containers (**podman generate kube**).
- **No Docker Compose built in** — Podman relies on a separate tool, **podman-compose**, or increasingly, native Compose-file support via **podman compose**, for multi-container orchestration equivalent to Chapter 9’s workflows.

11b.2 containerd and nerdctl

As covered in Chapter 11, **containerd** is the container runtime Docker itself uses under the hood, and is also used directly by Kubernetes. **nerdctl** is a Docker-CLI-compatible frontend for containerd, useful when you want Docker-like ergonomics on a system that only has containerd installed (common on Kubernetes nodes), including extra features like native support for lazy-pulling images and encrypted images.

11b.3 Buildah and Skopeo

Part of the same project family as Podman:

- **Buildah** — a tool focused purely on building OCI-compliant images, without needing a full container runtime; useful in CI pipelines that only need to build and push images, not run them.
- **Skopeo** — a tool for inspecting, copying, and signing container images and their metadata across registries, without needing to pull the full image or run a daemon.

11b.4 LXC/LXD

LXC (Linux Containers) predates Docker and provides lower-level, “system container” style virtualization — closer in spirit to a lightweight VM (running a full init system and multiple processes) than to Docker’s typical “one process per container” application-container model. **LXD**

is a management layer on top of LXC. These remain relevant for use cases wanting VM-like, full-system containers with lower overhead than true virtualization.

11b.5 gVisor and Kata Containers — Stronger Isolation Runtimes

Recall from Chapter 1 that the shared-kernel model is containers' main security trade-off versus VMs. Two projects address this directly, both implementing the OCI Runtime Specification (Chapter 11) so they can be used as drop-in alternatives to **runc**:

- **gVisor** (Google) — intercepts a container's system calls in a user-space sandbox, providing an additional isolation layer between the container and the host kernel, at some performance cost.
- **Kata Containers** — runs each container inside its own lightweight, fast-booting micro-VM, combining VM-grade isolation with container-like speed and density, at a greater resource cost than gVisor but with a smaller shared-kernel attack surface (each container effectively gets its own kernel again).

Both are commonly used for multi-tenant environments running untrusted workloads (e.g., public code-execution platforms, CI runners executing arbitrary user-submitted code), where the default shared-kernel isolation of standard **runc** containers is considered insufficient.

11b.6 Where Docker Fits Today

Despite this broader ecosystem, Docker remains the dominant on-ramp for learning containers and for local development workflows, thanks to its mature tooling (Docker Desktop, Docker Hub, Compose) and enormous documentation/community base — which is precisely why a course built around Docker, as this one is, remains the standard starting point, even though the concepts (images, layers, namespaces, cgroups, the OCI spec) generalize directly to every tool described in this chapter.

Chapter 12: Docker on Windows and macOS

12.1 Why This Needs Its Own Chapter

Docker containers require a Linux kernel — the namespaces and cgroups primitives described in Chapter 1 are Linux kernel features. Neither Windows nor macOS run a Linux kernel natively, so running Linux containers on either platform requires an additional virtualization layer. Understanding this layer explains a range of platform-specific quirks developers encounter.

12.2 Docker Desktop on macOS

Docker Desktop for Mac runs a small, purpose-built **Linux virtual machine** in the background (using Apple’s native `Virtualization.framework` on modern versions, replacing the older HyperKit-based approach). The actual `dockerd` daemon and all containers run **inside this VM**; the `docker` CLI on your Mac talks to that daemon over a socket that Docker Desktop transparently forwards.

12.2.1 Practical Implications

- **File sharing performance:** bind-mounting a macOS host directory into a Linux container crosses the VM boundary, historically causing noticeably slower file I/O than native Linux, especially for workloads with many small file reads (e.g., `node_modules`-heavy projects). Docker Desktop has invested heavily in optimizing this (via `VirtioFS`), substantially narrowing — though not eliminating — this gap.
- **Networking:** `localhost/127.0.0.1` on the Mac host is transparently forwarded into the VM for published container ports, so `docker run -p 8080:80` “just works” from the Mac’s perspective, even though it’s technically routing through the VM.
- **Resource allocation:** the VM’s CPU, memory, and disk allocation are configured via Docker Desktop’s Settings (Resources tab) — containers cannot use more resources than what’s been allocated to the underlying VM.
- **Apple Silicon (arm64) vs Intel (amd64):** Apple Silicon Macs run an `arm64` Linux VM. Pulling an `amd64`-only image will run under QEMU-based emulation (functional, but slower); many popular images now publish native `arm64` builds (multi-platform manifests) to avoid this.

12.3 Docker Desktop on Windows

Windows offers two historical backends; the modern, recommended one is **WSL2**.

12.3.1 WSL2 Backend (Recommended)

WSL2 (Windows Subsystem for Linux, version 2) runs a real, lightweight Linux kernel directly, managed by Windows’ own Hyper-V-based virtualization, but far more tightly integrated and efficient than a traditional VM. Docker Desktop installs its Linux daemon inside a dedicated WSL2 distribution.

- Any WSL2 Linux distribution (Ubuntu, Debian, etc.) installed alongside can be **enabled for Docker integration**, letting you run `docker` commands directly from a Linux shell with near-native performance.
- File performance is dramatically better when your project files live **inside the WSL2 filesystem** (e.g., `/home/user/project`) rather than on the Windows filesystem accessed via

`/mnt/c/...` — crossing between WSL2’s ext4 filesystem and Windows’ NTFS incurs overhead, so keeping the whole project in the Linux-side filesystem is a well-known performance best practice.

12.3.2 Hyper-V Backend (Legacy)

Before WSL2 matured, Docker Desktop for Windows ran a heavier, traditional Hyper-V virtual machine (similar in spirit to the Mac approach). It is largely superseded by WSL2 today but may still be selected in specific enterprise environments with WSL2 restrictions.

12.3.3 Windows Containers (Distinct From Linux Containers)

Confusingly, Docker Desktop on Windows can also run **native Windows containers** — containers built from Windows Server Core or Nano Server base images, using Windows’ own container isolation technology, rather than Linux namespaces/cgroups. These require switching Docker Desktop into “Windows containers” mode and are used specifically for containerizing legacy .NET Framework / Windows-only applications. Windows containers and Linux containers are **not interchangeable** — a Linux image cannot run as a Windows container and vice versa. The overwhelming majority of container workloads (and everything else in these notes) refers to Linux containers.

12.4 Common Cross-Platform Gotchas

Issue	Cause	Fix
Slow bind-mounted file I/O on Mac/Windows	Filesystem crosses the VM boundary	Keep project files inside WSL2’s Linux filesystem (Windows); rely on VirtioFS improvements (Mac); avoid bind-mounting huge dependency directories where possible
host networking behaves differently	<code>--network host</code> shares the VM’s network namespace on Mac/Windows, not literally the physical host’s	Prefer explicit port publishing (<code>-p</code>) for cross-platform consistency
Line-ending / permission mismatches	Windows files default to CRLF line endings; NTFS/WSL2 permission models differ from native Linux	Configure Git’s <code>core.autocrlf</code> ; keep scripts with a <code>.gitattributes</code> enforcing LF
Case sensitivity differences	macOS/Windows filesystems are typically case-insensitive by default; Linux (inside the container) is case-sensitive	Use consistent casing in <code>COPY/import</code> paths to avoid “works on my machine, fails in CI” surprises
Resource limits silently capping containers	Docker Desktop’s VM has a fixed CPU/RAM allocation	Increase allocation in Docker Desktop → Settings → Resources

12.5 When to Just Use Linux

For CI runners and production servers, Docker Engine is installed natively on Linux (Chapter 3), sidestepping the VM layer entirely — this is one of many reasons development environments (via Docker Desktop or a Linux VM/WSL2) aim to mirror production Linux as closely as possible, minimizing surprises when code moves from a developer’s Mac/Windows laptop to a Linux production host.

Chapter 13: Container Orchestration — Why and What

13.1 From One Container to a Fleet

Everything covered so far — running containers, networking them, persisting data, packaging multi-container apps with Compose — works well on a **single machine**. Production systems, however, typically need to run across **many machines**, for reasons including:

- **Capacity** — a single machine has finite CPU/RAM; a growing application needs to scale horizontally across multiple hosts.
- **High availability** — if an application runs on only one machine, that machine’s failure takes the application down entirely. Spreading replicas across multiple machines (and ideally multiple data centers/availability zones) avoids a single point of failure.
- **Rolling updates with zero downtime** — deploying a new version should replace old containers with new ones gradually, keeping the application available throughout.
- **Efficient scheduling** — given a pool of machines with varying available capacity, something needs to decide *which* machine each container should run on.

An **orchestrator** is the system responsible for all of this: scheduling containers onto machines, monitoring their health and restarting/rescheduling failed ones, handling scaling up/down, routing traffic to healthy instances, and coordinating rolling updates and rollbacks — all with minimal manual intervention.

13.2 What an Orchestrator Actually Does

Responsibility	Description
Scheduling	Decides which node (machine) in the cluster runs each container, based on resource availability and constraints
Service discovery & load balancing	Provides a stable way for services to find and route traffic to each other, even as individual container instances come and go
Self-healing	Detects failed containers/nodes and automatically restarts or reschedules affected workloads
Scaling	Increases or decreases the number of running replicas, manually or automatically based on load
Rolling updates & rollbacks	Replaces old container versions with new ones incrementally, and can revert automatically if health checks fail
Configuration & secrets management	Distributes configuration and sensitive credentials to the containers that need them
Storage orchestration	Coordinates attaching persistent storage to containers regardless of which node they’re scheduled on

13.3 The Two Major Orchestrators Covered in This Course

1. **Docker Swarm** — Docker’s own, built-in orchestrator. Simpler to learn and operate, tightly integrated with the Docker CLI and Compose file format, well suited to smaller clusters and teams already deeply invested in the Docker toolchain. (Chapter 14)
2. **Kubernetes** (often abbreviated **k8s**) — originally developed at Google, now a CNCF graduated project and the de facto industry-standard orchestrator, with a much larger feature set, ecosystem, and learning curve, dominant in enterprise and cloud-native environments. (Chapter 15)

Both solve the same fundamental problem; the practical decision of which to use usually comes down to team size/expertise, ecosystem requirements (Kubernetes has vastly more tooling, operators, and managed cloud offerings), and operational complexity tolerance.

13.4 A Cluster’s Building Blocks (Shared Vocabulary)

Regardless of orchestrator, a few concepts recur:

- **Node** — a single machine (physical or virtual) that’s part of the cluster.
- **Manager/control-plane node** — a node responsible for cluster-wide decisions: scheduling, maintaining cluster state, API access. (Swarm: “manager node”; Kubernetes: “control plane node.”)
- **Worker node** — a node that runs application workloads (containers) as directed by the control plane.
- **Service** — an abstraction over a group of identical, replicated container instances, providing stable networking/discovery even as individual instances are replaced.
- **Desired state reconciliation** — the core orchestration pattern: you declare the *desired* state (“I want 5 replicas of this container running”), and the orchestrator continuously works to reconcile the *actual* state with that desired state, automatically correcting drift (e.g., a crashed container is automatically replaced to get back to 5 running replicas).

This **declarative, desired-state** model — as opposed to manually and imperatively running commands to fix problems — is the defining philosophical shift orchestration introduces over plain `docker run`/Compose usage, and it’s worth internalizing before diving into Swarm and Kubernetes specifics.

Chapter 14: Docker Swarm

14.1 What Is Swarm?

Docker Swarm is Docker's native clustering and orchestration solution, built directly into the Docker Engine — no separate installation required. A group of Docker hosts (physical or virtual machines, running the Docker Engine) can be combined into a single logical **swarm**, which you then interact with using the same familiar **docker** CLI, extended with swarm-specific subcommands.

14.2 Initializing a Swarm

```
# On the first machine, which becomes the initial manager node:  
docker swarm init --advertise-addr <MANAGER_IP>
```

This command outputs a `docker swarm join` command (with a security token) that other machines use to join the swarm as **workers**:

```
# Run on each additional machine you want to add as a worker:  
docker swarm join --token <WORKER_TOKEN> <MANAGER_IP>:2377
```

To add another **manager** node (for control-plane high availability):

```
docker swarm join-token manager          # retrieve the manager join token  
docker swarm join --token <MANAGER_TOKEN> <MANAGER_IP>:2377
```

```
docker node ls                          # list all nodes in the swarm (run on a manager)  
docker node inspect NODE                 # detailed info about a node  
docker node update --availability drain NODE # cordon a node (stop  
                                              # scheduling new tasks on it,  
                                              # e.g. before maintenance)  
  
docker swarm leave                       # remove the current node from the swarm  
docker swarm leave --force                # forcibly remove a manager node
```

14.3 Manager Nodes and Raft Consensus

Manager nodes maintain the swarm's cluster state (which services should be running, their desired replica counts, current placements) using the **Raft consensus algorithm**, ensuring that all managers agree on cluster state even if some managers fail. For production high availability, an **odd number** of manager nodes (commonly 3 or 5) is recommended, since Raft requires a majority (quorum) to make decisions, and odd numbers avoid tie votes while tolerating the loss of $(N-1)/2$ managers without losing quorum.

Worker nodes do not participate in this consensus — they simply execute tasks assigned to them by managers. A manager node can also act as a worker (running application containers) unless explicitly drained.

14.4 Services — The Core Swarm Abstraction

Rather than running individual containers directly, Swarm introduces the **service** as its primary unit: a declarative specification of a desired container image, replica count, and configuration, which Swarm continuously reconciles.

```
docker service create \
  --name web \
  --replicas 3 \
  --publish 8080:80 \
  nginx:1.25
```

```
docker service ls           # list all services in the swarm
docker service ps web      # show individual tasks (container
                           # instances) for the "web" service
                           # and which node each runs on
docker service inspect web  # detailed configuration
docker service logs web     # aggregated logs across all replicas
```

14.4.1 Scaling a Service

```
docker service scale web=5
```

Swarm automatically schedules the two additional replicas onto nodes with available capacity, maintaining the desired count of 5 going forward — if any replica's container dies, Swarm automatically starts a replacement.

14.4.2 Rolling Updates

```
docker service update \
  --image nginx:1.26 \
  --update-parallelism 1 \
  --update-delay 10s \
  web
```

Swarm replaces replicas incrementally (one at a time here, waiting 10 seconds between each), rather than all at once, keeping the service available throughout the rollout. If health checks fail during rollout, Swarm can be configured to automatically roll back:

```
docker service update --rollback web      # manual rollback to the
                                           # previous version
```

```
# In a stack file (see 14.6):
deploy:
  update_config:
    parallelism: 1
    delay: 10s
    failure_action: rollback
    monitor: 30s
```

14.5 The Routing Mesh and Load Balancing

When a service publishes a port, Swarm's **ingress routing mesh** automatically makes that service reachable on that port from **every node in the swarm** — even nodes not currently running any replica of that service — transparently routing incoming traffic to a node that *is* running a healthy

replica. This is combined with internal load-balancing (via an embedded DNS-based virtual IP) when one service calls another by name, spreading requests across all healthy replicas automatically.

14.6 Stacks — Deploying Compose Files to Swarm

A **stack** is Swarm’s mechanism for deploying a multi-service application described in a Compose-file-compatible YAML format (a “stack file”) in one command — bridging directly from the Compose knowledge in Chapter 9.

```
# docker-stack.yml
version: "3.9"
services:
  web:
    image: myrepo/web:1.0
    ports:
      - "8080:80"
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
    networks:
      - appnet

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - dbdata:/var/lib/postgresql/data
    deploy:
      replicas: 1
      placement:
        constraints: [node.role == manager]
    networks:
      - appnet

volumes:
  dbdata:

networks:
  appnet:
    driver: overlay
```

```
docker stack deploy -c docker-stack.yml myapp
docker stack ls
```

```
docker stack services myapp
docker stack ps myapp
docker stack rm myapp
```

Note the `deploy:` key, which is specific to Swarm (and ignored by plain `docker compose up`), and the `overlay` network driver (introduced in Chapter 7), which is what actually enables containers on **different physical nodes** to communicate as if on the same network.

14.7 Secrets and Configs

Swarm provides first-class, encrypted-at-rest primitives for distributing sensitive data and configuration files to services, superior to plain environment variables for secrets:

```
echo "supersecretpassword" | docker secret create db_password -
docker secret ls
```

```
services:
  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD_FILE=/run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    external: true
```

Secrets are mounted as in-memory files inside the container's filesystem (at `/run/secrets/<name>` by default), never as plain environment variables, and are only delivered to nodes actually running a task that references them.

14.8 Placement Constraints and Node Labels

```
docker node update --label-add zone=us-east node1
```

```
deploy:
  placement:
    constraints:
      - node.labels.zone == us-east
      - node.role == worker
```

Placement constraints let you control which nodes are eligible to run a given service's tasks — useful for pinning stateful services to specific nodes with attached storage, or spreading replicas across failure domains.

14.9 When Swarm Is a Good Fit

Swarm shines when a team already knows Docker and Compose well, needs multi-host orchestration without a steep learning curve, and doesn't require Kubernetes' broader ecosystem (custom

resources, extensive operator ecosystem, fine-grained RBAC, etc.). Many small-to-medium production deployments run happily on Swarm for years. Kubernetes' dominance in large enterprises and cloud-native tooling, however, means Swarm's ecosystem and community momentum have shrunk considerably relative to Kubernetes over time — a practical consideration when choosing for a new, long-lived project.

Chapter 15: Kubernetes Fundamentals

15.1 What Is Kubernetes and Why Learn It Here

Kubernetes (from the Greek for “helmsman,” commonly abbreviated **k8s** — “k,” 8 letters, “s”) is an open-source container orchestration platform originally designed at Google (drawing on their internal system, Borg), donated to the **Cloud Native Computing Foundation (CNCF)** in 2015, and now the dominant orchestrator in the industry. Even though this course centers on Docker, understanding Kubernetes fundamentals is essential context: Docker-built images are exactly what Kubernetes runs, and Kubernetes has effectively become the default target for “deploying containerized apps” at any meaningful scale — the very phrase in this course’s title.

Kubernetes is considerably more feature-rich (and more complex to operate) than Docker Swarm, with a correspondingly larger vocabulary of concepts.

15.2 Kubernetes Architecture

A Kubernetes cluster has two categories of nodes:

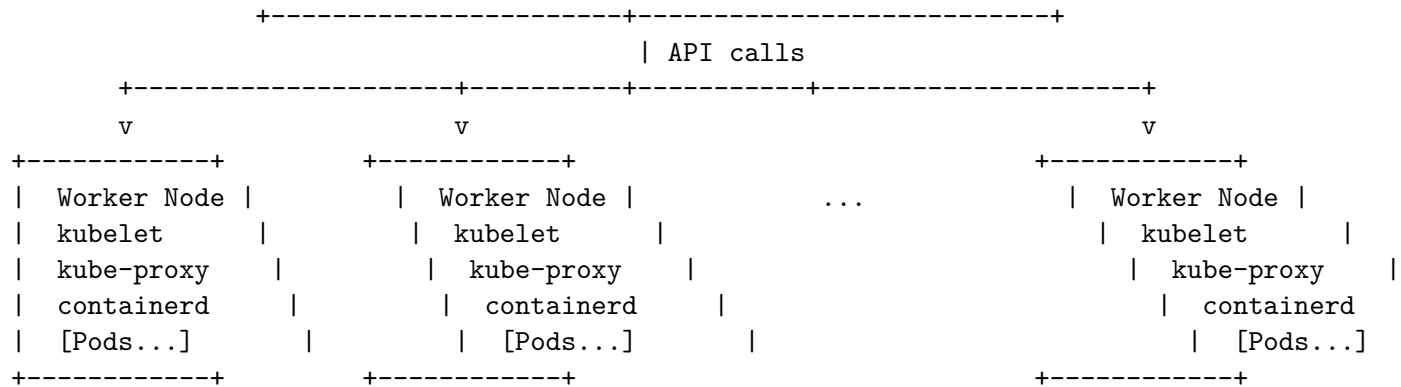
15.2.1 Control Plane Components

- **kube-apiserver** — the front door to the cluster; exposes the Kubernetes REST API, which every other component (and **kubectl**) talks to.
- **etcd** — a distributed, consistent key-value store holding the entire cluster’s state (the single source of truth for desired configuration).
- **kube-scheduler** — watches for newly created Pods with no assigned node, and selects a suitable node for each, based on resource requirements, constraints, and affinity/anti-affinity rules.
- **kube-controller-manager** — runs controller processes that continuously drive actual cluster state toward the desired state described in **etcd** (e.g., the “ReplicaSet controller” ensures the right number of Pod replicas exist).
- **cloud-controller-manager** — integrates with the underlying cloud provider (AWS, GCP, Azure) for cloud-specific resources like load balancers and storage volumes.

15.2.2 Node (Worker) Components

- **kubelet** — an agent running on every node, responsible for ensuring containers described in assigned Pods are actually running and healthy, communicating with the container runtime.
- **kube-proxy** — maintains network rules on each node, implementing the Service abstraction’s load balancing (Section 15.5).
- **Container runtime** — the software that actually runs containers, accessed via the **Container Runtime Interface (CRI)**. Modern Kubernetes clusters typically use **containerd** or **CRI-O** directly (recall from Chapter 2 that containerd is the same component Docker itself uses under the hood) — note that as of Kubernetes 1.24+, the old direct “dockershim” integration was removed, meaning Kubernetes no longer talks to the full Docker Engine, only to an OCI-compliant runtime like containerd. Docker-built images remain fully compatible, since images are a standardized OCI format regardless of which runtime executes them.

```
+----- Control Plane -----+
| kube-apiserver -- etcd -- scheduler -- controller-mgr |
```



15.3 The Pod — Kubernetes’ Smallest Deployable Unit

Notably, Kubernetes does not schedule individual containers directly — its smallest deployable unit is the **Pod**, a group of one or more containers that are always scheduled together onto the same node, share the same network namespace (and therefore the same IP address and port space, communicating with each other over `localhost`), and can share storage volumes.

Most Pods run a single container; multi-container Pods are used for tightly coupled helper patterns:

- **Sidecar** — a helper container running alongside the main one (e.g., a log-shipping agent).
- **Init containers** — containers that run to completion *before* the main application container starts, commonly used for setup tasks (waiting for a dependency, running a database migration).

```

# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp
    image: myrepo/myapp:1.0
    ports:
      - containerPort: 8080
  resources:
    requests:
      cpu: "250m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "256Mi"

```

```

kubectl apply -f pod.yaml
kubectl get pods
kubectl describe pod myapp-pod

```

```
kubectl logs myapp-pod
kubectl exec -it myapp-pod -- bash
kubectl delete pod myapp-pod
```

In practice, **Pods are rarely created directly** — they're ephemeral and, on their own, offer no self-healing (a directly-created Pod that dies is simply gone). Instead, higher-level controllers (Deployments, StatefulSets, etc.) manage Pods on your behalf.

15.4 Deployments — Managing Replicated, Self-Healing Pods

A **Deployment** declares a desired Pod template and replica count; Kubernetes continuously ensures that many healthy copies of that Pod are running, replacing any that fail, and manages rolling updates when the Pod template changes.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myrepo/myapp:1.0
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
```

```
    port: 8080
    initialDelaySeconds: 15
    periodSeconds: 20
```

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get replicaset           # Deployments create/manage ReplicaSets,
                                # which in turn create/manage Pods
kubectl scale deployment myapp --replicas=5
kubectl set image deployment/myapp myapp=myrepo/myapp:1.1 # trigger a
                                                            # rolling update

kubectl rollout status deployment/myapp
kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp # rollback to the previous revision
```

15.4.1 Readiness vs. Liveness Probes

- **Readiness probe** — determines whether a Pod is ready to *receive traffic*. A Pod failing its readiness probe is removed from a Service’s load-balancing pool, but not killed — useful during slow startup or temporary overload.
- **Liveness probe** — determines whether a Pod is still *healthy*. A Pod failing its liveness probe is killed and restarted by the kubelet.

This distinction — familiar in spirit to the `depends_on: condition: service_healthy` pattern in Compose (Chapter 9) and Docker’s `HEALTHCHECK` (Chapter 6) — is central to Kubernetes’ self-healing behavior.

15.5 Services — Stable Networking for Ephemeral Pods

Because Pods are ephemeral (a Deployment might replace them at any time, each getting a new internal IP address), Kubernetes provides the **Service** abstraction: a stable virtual IP address and DNS name that load-balances across whichever Pods currently match a given label selector.

```
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

15.5.1 Service Types

Type	Behavior
ClusterIP (default)	Internal-only virtual IP, reachable from within the cluster
NodePort	Additionally exposes the service on a static port on every node's IP, making it reachable from outside the cluster
LoadBalancer	Provisions an external load balancer via the cloud provider (e.g., an AWS ELB), giving the service a public IP
ExternalName	Maps the service to an external DNS name, for referring to services outside the cluster by an internal name

```
kubectl apply -f service.yaml
kubectl get services
kubectl expose deployment myapp --port=80 --target-port=8080 --type=LoadBalancer
```

15.6 Ingress — HTTP(S) Routing Into the Cluster

While a `LoadBalancer` Service works for exposing a single service, real applications typically need path- and host-based HTTP routing across many services (e.g., `api.example.com` → the API service, `example.com/admin` → the admin service), plus centralized TLS termination. This is the job of an **Ingress** resource, interpreted by an **Ingress Controller** (e.g., NGINX Ingress Controller, Traefik) running in the cluster.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: myapp-service
                port:
                  number: 80
  tls:
    - hosts: ["myapp.example.com"]
      secretName: myapp-tls
```

15.7 ConfigMaps and Secrets

Analogous to Compose’s `environment/env_file` (Chapter 9) and Swarm’s secrets (Chapter 14), Kubernetes externalizes configuration and sensitive data from container images:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: myapp-config
data:
  APP_ENV: production
  LOG_LEVEL: info
---
apiVersion: v1
kind: Secret
metadata:
  name: myapp-secret
type: Opaque
stringData:
  DB_PASSWORD: supersecret

# referenced inside a Pod spec:
envFrom:
- configMapRef:
    name: myapp-config
- secretRef:
    name: myapp-secret
```

15.8 Namespaces

Namespaces partition a single cluster into multiple virtual clusters, commonly used to separate environments (`dev`, `staging`, `production`) or teams within a shared physical cluster, each with their own resource quotas and, typically, role-based access control (RBAC) boundaries.

```
kubectl create namespace staging
kubectl apply -f deployment.yaml -n staging
kubectl get pods -n staging
kubectl config set-context --current --namespace=staging # switch default
                                                         # namespace for
                                                         # subsequent commands
```

15.9 Persistent Storage

Kubernetes decouples storage requests from storage provisioning via two resources: a **PersistentVolume (PV)** represents an actual piece of storage (provisioned by an administrator or dynamically by a cloud provider’s storage class), and a **PersistentVolumeClaim (PVC)** is a request for storage by a Pod, which Kubernetes binds to a matching PV.

```
apiVersion: v1
kind: PersistentVolumeClaim
```

```

metadata:
  name: myapp-data
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard

# referenced inside a Pod spec:
volumes:
- name: data
  persistentVolumeClaim:
    claimName: myapp-data
containers:
- name: myapp
  volumeMounts:
  - mountPath: /data
    name: data

```

15.10 kubectl Quick Reference

```

kubectl get pods,deployments,services,nodes      # list resources
kubectl get pods -o wide                         # more detail, including node placement
kubectl describe pod POD_NAME                   # detailed status and recent events
kubectl logs POD_NAME [-c CONTAINER]            # logs (specify container in multi-container)
kubectl exec -it POD_NAME -- sh                 # shell into a Pod
kubectl apply -f FILE.yaml                      # create/update resources declaratively
kubectl delete -f FILE.yaml                    # remove resources
kubectl get events --sort-by=.metadata.creationTimestamp # cluster events, newest last
kubectl top pods                                # resource usage (requires metrics)
kubectl config get-contexts                      # list available cluster contexts
kubectl config use-context CONTEXT_NAME          # switch cluster context

```

15.11 Helm — Packaging Kubernetes Applications

Real-world Kubernetes applications often require many interrelated YAML files (Deployments, Services, ConfigMaps, Ingress, etc.). **Helm** is the de facto package manager for Kubernetes, letting applications be packaged as versioned, parameterized **charts**, and installed/upgraded/rolled back as a single unit — conceptually similar in spirit to how Compose bundles multiple services into one file, but for Kubernetes and with templating support.

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-postgres bitnami/postgresql
helm upgrade my-postgres bitnami/postgresql --set auth.password=newpass
helm rollback my-postgres 1
helm uninstall my-postgres

```

15.12 Local Kubernetes for Development and Learning

Running a full multi-node cluster isn't necessary to learn or develop against Kubernetes; several tools provide a lightweight, single-machine cluster:

- **Docker Desktop's built-in Kubernetes** — one-click enable, runs a single-node cluster inside Docker Desktop's existing VM.
- **Minikube** — a dedicated single-node cluster tool, widely used for local development and tutorials.
- **kind** (Kubernetes IN Docker) — runs Kubernetes nodes themselves as Docker containers, popular for fast, disposable clusters (especially in CI pipelines for testing Kubernetes manifests).
- **k3s** / **k3d** — a lightweight, certified Kubernetes distribution (k3s) well suited to edge/resource-constrained environments, with **k3d** providing a Docker-based wrapper similar to **kind**.

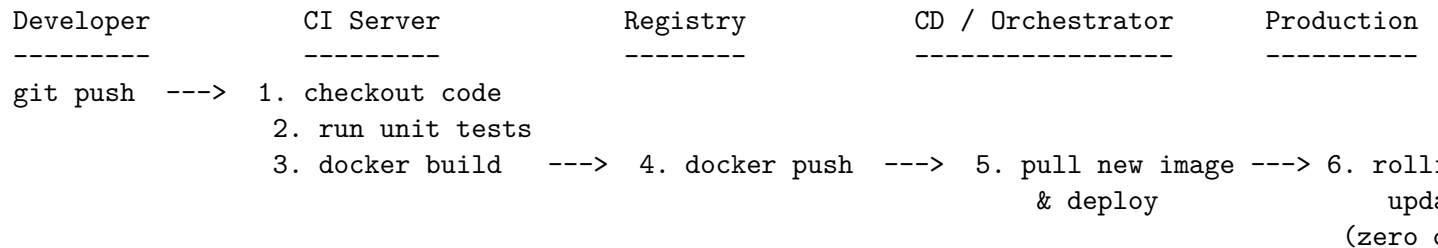
15.13 Docker Swarm vs. Kubernetes — Summary Comparison

Aspect	Docker Swarm	Kubernetes
Learning curve	Gentle; reuses Docker CLI/Compose knowledge	Steep; large, distinct vocabulary and tooling
Setup complexity	Simple (<code>docker swarm init</code>)	Significant (though managed cloud offerings — EKS, GKE, AKS — greatly simplify this)
Ecosystem & tooling	Smaller, less active	Vast (Helm charts, operators, service meshes, extensive CNCF ecosystem)
Scalability	Good for small/medium clusters	Proven at very large scale
Built into Docker Engine	Yes	No — separate installation/distribution
Industry adoption trend	Declining	Dominant, still growing
Best fit	Small teams, simpler deployments, fast onboarding	Larger organizations, complex multi-service systems, cloud-native ecosystems

Chapter 16: CI/CD and Deploying Containerized Applications

16.1 The Full Pipeline, End to End

This chapter ties together everything from prior chapters into a complete picture: how a code change goes from a developer's commit to running safely in production, using containers as the deployment artifact throughout.



16.2 Continuous Integration: Building and Testing the Image

A typical CI job (GitHub Actions, GitLab CI, Jenkins) for a containerized application:

```
# .github/workflows/ci.yml (GitHub Actions example)
name: CI
on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run tests
        run: docker build --target test -t myapp:test .
        # (assumes a multi-stage Dockerfile with a "test" stage that runs
        # the test suite and fails the build on test failure)

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}

      - name: Build and push image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
```

```
tags: |
  ghcr.io/myorg/myapp:${{ github.sha }}
  ghcr.io/myorg/myapp:latest
cache-from: type=gha
cache-to: type=gha,mode=max
```

Key patterns worth internalizing:

- **Tag by commit SHA**, not just `latest` — this gives every build a unique, traceable, immutable identifier linking a running container straight back to the exact source commit it came from (Chapter 10).
- **Registry build cache** (`cache-from/cache-to`) speeds up CI builds by reusing layers across runs, leveraging BuildKit’s registry-cache export (Chapter 11).
- **A dedicated test build stage** in a multi-stage Dockerfile (Chapter 6) lets the exact same Dockerfile used for production also run the test suite in CI, keeping build logic in one place.

16.3 Continuous Deployment: Rolling Out the New Image

Once a new image is pushed to the registry, the deployment step differs by target platform:

16.3.1 Deploying to Docker Swarm

```
docker service update --image ghcr.io/myorg/myapp:$GIT_SHA myapp_web
```

16.3.2 Deploying to Kubernetes

```
kubectl set image deployment/myapp myapp=ghcr.io/myorg/myapp:$GIT_SHA
kubectl rollout status deployment/myapp
```

Or, more commonly in mature setups, via **GitOps**: a tool like **ArgoCD** or **Flux** continuously watches a Git repository containing the desired Kubernetes manifests (including the image tag), and automatically applies any changes to the cluster — meaning “deployment” becomes simply “commit the new image tag to the manifests repo,” with the actual cluster reconciliation handled declaratively and auditable entirely through Git history.

16.3.3 Deploying to a Single Server (Simple Cases)

For smaller applications not needing a full orchestrator, a deployment step might simply SSH into a server and pull/restart:

```
ssh deploy@server "docker pull ghcr.io/myorg/myapp:$GIT_SHA && \
  docker compose -f /opt/myapp/docker-compose.yml up -d"
```

16.4 Deployment Strategies

Strategy	Description	Trade-off
Recreate	Stop all old instances, then start new ones	Simple but causes downtime
Rolling update	Replace instances incrementally (Chapters 14–15)	No downtime, but old and new versions briefly coexist
Blue-Green	Deploy the new version fully alongside the old, then switch traffic all at once	Instant rollback (switch traffic back); requires double the resources temporarily
Canary	Route a small percentage of traffic to the new version first, gradually increasing	Limits blast radius of bad deploys; requires more sophisticated traffic routing

Blue-green and canary deployments typically require an orchestrator with fine-grained traffic-splitting support (Kubernetes with a service mesh like Istio/Linkerd, or a cloud load balancer’s weighted routing) beyond what plain Compose or basic Swarm rolling updates provide out of the box.

16.5 Health Checks Tie the Whole Pipeline Together

The `HEALTHCHECK` instruction (Chapter 6), Compose’s `healthcheck/condition: service_healthy` (Chapter 9), Swarm’s health-aware routing mesh (Chapter 14), and Kubernetes’ readiness/liveness probes (Chapter 15) are all the same underlying idea applied at different layers: **the deployment tooling needs to know when a container is actually ready to serve traffic**, not just “started.” A robust CI/CD pipeline depends on accurate health signals to safely automate rolling updates and rollbacks.

16.6 Observability in Production

Once deployed, containerized applications need to be observable:

- **Logs** — collected via a log driver (Chapter 11) or a sidecar/DaemonSet log-shipping agent, centralized into a system like the ELK/EFK stack (Elasticsearch/OpenSearch + Fluentd/Logstash + Kibana) or a managed service (Datadog, CloudWatch Logs).
- **Metrics** — container/application metrics scraped by **Prometheus** and visualized in **Grafana**; `docker stats` and `kubectl top` are the manual, ad hoc equivalents (Chapters 4 and 15).
- **Tracing** — distributed tracing (OpenTelemetry, Jaeger) tracks a single request’s path across multiple microservice containers, essential for diagnosing latency issues in complex systems.
- **Alerting** — rules on top of metrics/logs (e.g., via Prometheus Alertmanager) notify on-call engineers of problems before users report them.

Chapter 16b: Capstone Project — Containerizing and Deploying a Three-Tier Application

This chapter walks through a complete, realistic example end to end, applying nearly every concept from the preceding chapters to one project: a **three-tier web application** consisting of a React frontend, a Node.js/Express API, and a PostgreSQL database, taken from local development all the way to a deployed, orchestrated production setup.

16b.1 Project Layout

```
myapp/
+-- frontend/
|   +-- Dockerfile
|   +-- package.json
|   +-- src/
+-- api/
|   +-- Dockerfile
|   +-- package.json
|   +-- src/
+-- docker-compose.yml
+-- docker-compose.override.yml
+-- docker-stack.yml
+-- .dockerignore
+-- .env.example
```

16b.2 The API Dockerfile

Applying multi-stage builds (Chapter 6), layer-cache-friendly instruction ordering, a non-root user (Chapter 17), and an ENTRYPOINT/CMD split (Chapter 6):

```
# api/Dockerfile
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS production
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
ENV NODE_ENV=production
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
```

```

COPY package*.json ./
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD node -e "require('http').get('http://localhost:5000/health', r => process.exit(r.statusCode))"
ENTRYPOINT ["node"]
CMD ["dist/server.js"]

```

16b.3 The Frontend Dockerfile

```

# frontend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.25-alpine AS production
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=30s CMD wget -qO- http://localhost/ || exit 1
CMD ["nginx", "-g", "daemon off;"]

```

16b.4 Local Development with Compose

```

# docker-compose.yml (base, environment-agnostic)
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - api
    networks: [appnet]

  api:
    build: ./api
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgres://postgres:${DB_PASSWORD}@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy

```

```

networks: [appnet]

db:
  image: postgres:16
  environment:
    - POSTGRES_PASSWORD=${DB_PASSWORD}
    - POSTGRES_DB=appdb
  volumes:
    - dbdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    retries: 5
  networks: [appnet]

volumes:
  dbdata:

networks:
  appnet:
    driver: bridge

```

docker-compose.override.yml (auto-loaded locally)

```

services:
  frontend:
    build:
      target: build
    volumes:
      - ./frontend/src:/app/src
    command: npm run dev
  api:
    build:
      target: build
    volumes:
      - ./api/src:/app/src
    command: npm run dev
    environment:
      - NODE_ENV=development

```

```

cp .env.example .env           # fill in DB_PASSWORD, etc.
docker compose up --build      # local dev loop, live-reloading via bind mounts

```

16b.5 CI: Build, Test, Push

```

# .github/workflows/ci.yml
name: CI
on: { push: { branches: [main] } }

```

```

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix: { service: [frontend, api] }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: "${{ github.actor }}", password: "${{ secrets.GIT_TOKEN }}" }
      - uses: docker/build-push-action@v5
        with:
          context: ./${{ matrix.service }}
          push: true
          tags: ghcr.io/myorg/myapp-${{ matrix.service }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

```

16b.6 Production Deployment: Docker Swarm Stack

```

# docker-stack.yml
version: "3.9"
services:
  frontend:
    image: ghcr.io/myorg/myapp-frontend:${IMAGE_TAG}
    ports: ["80:80"]
    deploy:
      replicas: 2
      update_config: { parallelism: 1, delay: 10s, failure_action: rollback }
    networks: [appnet]

  api:
    image: ghcr.io/myorg/myapp-api:${IMAGE_TAG}
    deploy:
      replicas: 3
      update_config: { parallelism: 1, delay: 10s, failure_action: rollback }
      resources: { limits: { cpus: "0.5", memory: 256M } }
    secrets: [db_password]
    environment:
      - DATABASE_URL=postgres://postgres@db:5432/appdb
    networks: [appnet]

  db:
    image: postgres:16
    deploy:
      replicas: 1
      placement: { constraints: ["node.role == manager"] }
    secrets: [db_password]

```

```

    volumes: ["dbdata:/var/lib/postgresql/data"]
    networks: [appnet]

secrets:
  db_password:
    external: true

volumes:
  dbdata:

networks:
  appnet:
    driver: overlay

docker secret create db_password db_password.txt
IMAGE_TAG=$(git rev-parse --short HEAD) docker stack deploy -c docker-stack.yml myapp
docker stack services myapp

```

16b.7 What This Project Demonstrates

- Multi-stage builds producing lean, secure production images (Chapter 6)
- Non-root users and healthchecks baked into images (Chapters 6, 17)
- User-defined networking with name-based service discovery (Chapter 7)
- Persistent, isolated database storage via a named volume (Chapter 8)
- A clean local-dev/production split using Compose file layering (Chapter 9)
- Immutable, digest-traceable image tags built in CI (Chapters 10, 16)
- Zero-downtime rolling deployment with automatic rollback on failure, secrets management, and placement constraints via Docker Swarm (Chapter 14)

The exact same application, packaged the same way, could equally be deployed to Kubernetes by translating the `docker-stack.yml` into a Deployment + Service + Ingress + Secret set, following the patterns from Chapter 15 — illustrating that once an application is properly containerized, the choice of orchestrator becomes a deployment-time decision rather than something baked into the application itself.

Chapter 17: Best Practices, Security, and Common Pitfalls

17.1 Dockerfile Best Practices Checklist

- **Use specific, minimal base images** — pin exact versions (`python:3.12.4-slim`, not `python:latest`), and prefer slim/alpine/distroless variants when compatible, to minimize both image size and attack surface (Chapter 6).
- **Order instructions from least to most frequently changing** — dependency installation before source code copy, to maximize layer cache hits (Chapter 6).
- **Use multi-stage builds** to keep build tools and intermediate artifacts out of the final production image (Chapter 6).
- **Use a `.dockerignore` file** to keep the build context small and avoid leaking secrets/dev files into the image (Chapter 6).
- **Prefer exec-form CMD/ENTRYPOINT** (`["executable", "arg"]`) over shell form, so your application process receives signals directly and shuts down gracefully (Chapter 6).
- **Run as a non-root user** whenever possible:

```
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser
```

- **Set explicit resource requests/limits** at deploy time (`--memory`, `--cpus`, or their Compose/Kubernetes equivalents) so a single misbehaving container can't starve others on shared infrastructure.
- **Add a HEALTHCHECK** so orchestrators (and `docker ps`) can accurately determine container health (Chapter 6).
- **Never bake secrets into image layers** — not via `ENV`, not via a `COPY`'d credentials file that's later deleted (deletion doesn't remove it from earlier layers!). Use build secrets (`--secret`), runtime environment variables, or a secrets manager instead (Chapters 5, 11, 14, 15).

17.2 Image Security

17.2.1 Minimize the Attack Surface

Every package installed in an image is a potential vulnerability. Slim/alpine/distroless base images, combined with only installing what's strictly necessary (`--no-install-recommends` on Debian/Ubuntu, cleaning up package manager caches within the same `RUN` layer), keeps the surface small:

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*
```

17.2.2 Scan Images for Known Vulnerabilities

Integrate vulnerability scanning (Docker Scout, Trivy, Snyk, or a registry's built-in scanner — Chapter 10) into CI, ideally **failing the build** on critical/high-severity findings in a base image or dependency, rather than discovering them post-deployment.

```
trivy image myapp:1.0
```

17.2.3 Don't Run Containers as Root Unnecessarily

Running as root inside a container is not automatically catastrophic (namespaces still provide isolation), but it removes a defense-in-depth layer: a container-escape vulnerability combined with a root-owned container process is strictly worse than the same vulnerability combined with an unprivileged process. Combine a non-root USER with **user namespace remapping** (Chapter 11) for defense in depth.

17.2.4 Avoid `--privileged` and Unnecessary Capabilities

```
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE myapp
```

`--privileged` grants a container essentially all capabilities and direct device access — appropriate only for specific infrastructure tooling (e.g., containers that need to manage other containers, or interact with host hardware directly), never for typical application workloads. Prefer dropping all capabilities and adding back only the specific ones required.

17.2.5 Read-Only Root Filesystems

```
docker run --read-only --tmpfs /tmp myapp
```

Mounting the container's root filesystem read-only (with an explicit writable `tmpfs` for genuinely required scratch space) prevents an attacker who gains code execution inside the container from persisting malicious files or tampering with the application binary.

17.2.6 Keep the Docker Engine and Host OS Patched

Vulnerabilities in the container runtime itself (`runc`, `containerd`, the daemon) or the host kernel can, in rare but serious cases, allow container escape. Regular patching of the host and Docker Engine is as important as patching application dependencies.

17.3 Networking and Secrets Hygiene

- Bind services to the minimal necessary network interface (avoid publishing ports you don't need to expose externally).
- Use user-defined networks to segment unrelated applications from each other, rather than putting everything on the default bridge (Chapter 7).
- Store secrets in a dedicated secrets manager (Swarm secrets, Kubernetes Secrets backed by an external provider like Vault, or a cloud provider's secrets service) rather than plain environment variables or `.env` files in production (Chapters 5, 14, 15).
- Rotate credentials regularly, and ensure secrets are never logged or included in error messages.

17.4 Operational Best Practices

- **Set log rotation limits** on the `json-file` driver (or ship logs off-host entirely) to avoid disk exhaustion (Chapter 11).
- **Define resource requests/limits** for every service in production, both to protect against noisy neighbors and to give the scheduler (Swarm/Kubernetes) the information it needs to place workloads efficiently.

- **Use immutable, digest-pinned deployments** for production releases, resolving a tag to its digest at deploy time for full reproducibility (Chapter 10).
- **Automate everything** — manual `docker run` commands are fine for learning and local development, but production deployments should flow through version-controlled Compose/stack/Kubernetes manifests and a CI/CD pipeline (Chapter 16), never ad hoc commands run by hand against a production host.
- **Practice graceful shutdown** — ensure your application handles `SIGTERM` properly (finishing in-flight requests, closing connections cleanly) within the grace period Docker/Kubernetes allow before sending `SIGKILL` (default 10s in Docker, configurable).
- **Regularly prune unused resources** (Chapter 4) on long-lived hosts to reclaim disk space, but never blindly run `--volumes` pruning in production without confirming nothing important is unreferenced.

17.5 Common Beginner Mistakes, Revisited

Mistake	Why it's a problem	Fix
Using <code>latest</code> tag in production	Non-reproducible; silent behavior changes	Pin explicit versions or digests
Storing state in the container's writable layer	Data lost on container removal/recreation	Use volumes (Chapter 8)
Running everything as root	Weakens the security boundary	Add a non-root <code>USER</code> (this chapter)
One giant Dockerfile installing everything into one stage	Bloated final image, slow builds	Multi-stage builds (Chapter 6)
Hardcoding configuration in application code	Can't promote the same image across environments	Externalize via environment variables (Chapter 5)
Ignoring <code>.dockerignore</code>	Slow builds, possible secret leakage into image	Add a thorough <code>.dockerignore</code> (Chapter 6)
No health checks	Orchestrators can't tell if a container is actually working	Add <code>HEALTHCHECK</code> /readiness probes (Chapters 6, 15)
Manually SSHing in to deploy	Error-prone, unrepeatable, not auditable	Automate via CI/CD (Chapter 16)

Chapter 17b: Troubleshooting Docker — A Practical Field Guide

Even with a solid conceptual foundation, real-world Docker usage inevitably runs into friction. This chapter collects the most common failure modes, how to diagnose them, and how to fix them — organized by symptom, the way you’d actually encounter them.

17b.1 “Cannot connect to the Docker daemon”

Cannot connect to the Docker daemon at `unix:///var/run/docker.sock`.

Is the docker daemon running?

Diagnosis steps:

```
sudo systemctl status docker      # is the daemon actually running?
sudo systemctl start docker       # start it if not
ls -la /var/run/docker.sock      # check socket exists and permissions
groups $USER                     # is your user in the "docker" group?
```

Common causes: the daemon crashed or was never started; your user isn’t in the `docker` group (Chapter 3); Docker Desktop isn’t running (Mac/Windows); the daemon is configured to listen on a different socket/port than the client expects.

17b.2 Container Exits Immediately After `docker run`

```
docker run -d myimage
docker ps -a      # STATUS shows "Exited (0)" or "Exited (1)" seconds ago
```

Diagnosis:

```
docker logs CONTAINER      # what did the process print before exiting?
docker inspect CONTAINER --format '{{.State.ExitCode}}'
```

Common causes:

- The container’s main process (PID 1) finished its work and returned — this is expected for one-shot commands (`sleep 5`, `echo hi`) but not for long-running services.
- Missing `-it` when trying to run an interactive shell as the main process, so there’s no persistent foreground process to keep the container alive (Chapter 4).
- The application crashed on startup — check `docker logs` for a stack trace; common causes include missing environment variables (Chapter 5), an unreachable dependency (database not ready — Chapter 9’s `depends_on`/healthcheck discussion), or a missing file that should have been COPY’d in the Dockerfile.

17b.3 “Port is already allocated”

Error response from daemon: driver failed programming external connectivity on endpoint web: Bind for 0.0.0.0:8080 failed: port is already allocated

Diagnosis:

```
docker ps --filter "publish=8080"    # is another container using it?
sudo lsof -i :8080                    # is a non-Docker process using it? (Linux/Mac)
```

Fix: stop the conflicting container/process, or choose a different host port (`-p 8081:80`).

17b.4 Container Can't Reach Another Container by Name

Covered in depth in Chapter 7, but as a troubleshooting checklist:

1. Are both containers attached to the **same** network? `docker network inspect NETWORK` and check the `Containers` list.
2. Is it the **default** bridge network? Name resolution doesn't work there — create/use a user-defined network.
3. Is the target application actually listening on `0.0.0.0`, not `127.0.0.1`, inside its container?
4. Is the target container actually healthy and listening yet (a race condition at startup)? Add a proper healthcheck and `depends_on: condition: service_healthy` (Chapter 9), or implement retry logic in the client application.

17b.5 “No space left on device”

Docker's storage area filling the host disk is one of the most common production incidents.

```
docker system df          # see what's consuming space
docker system df -v       # verbose, per-object breakdown
docker system prune -a --volumes # aggressive cleanup (CAUTION: deletes
                                # all unused images, containers,
                                # networks, AND volumes)
```

Root causes to investigate, beyond one-time cleanup: unbounded container log growth (set `log-opts` limits — Chapter 11), CI runners accumulating build cache/images without periodic pruning, or genuinely needing more disk allocated to the Docker storage area (or, on Mac/Windows, to the Docker Desktop VM's virtual disk).

17b.6 Build Is Slow or Cache Never Hits

Revisit Chapter 6's caching discussion: instruction order matters. Diagnostic steps:

```
docker build --progress=plain -t myapp .    # see exactly which layers rebuild
```

Common causes: `COPY . .` placed before dependency installation (invalidates cache on every source change); no `.dockerignore`, so irrelevant file changes (like editor swap files or `.git` history) invalidate the build context hash; using `--no-cache` unnecessarily; base image itself changed upstream (if using a mutable tag like `:latest`).

17b.7 “Permission denied” Inside a Container

Often arises from bind-mounting host directories with UID/GID mismatches (Chapter 8), especially when the image runs as a non-root user (Chapter 17) with a different UID than the host user who owns the mounted files.

```
id -u          # your host UID
docker run --user $(id -u):$(id -g) -v $(pwd):/app myimage # match UIDs
```

Alternatively, adjust the Dockerfile to create the container's user with a matching UID, or fix host directory ownership/permissions directly.

17b.8 Image Builds Fine Locally but Fails in CI

Common causes: CI build context differs from local (e.g., `.dockerignore` excludes something CI needs, or a `.env` file present locally is absent in CI and not committed); platform/architecture mismatch (building `arm64` locally on Apple Silicon, but CI runners are `amd64` — use `docker buildx build --platform` explicitly, Chapter 11); CI lacking credentials/network access to pull a private base image; non-deterministic build steps (e.g., pulling `:latest` of a dependency that changed).

17b.9 Container Runs Out of Memory (OOMKilled)

```
docker inspect CONTAINER --format='{{.State.OOMKilled}}'      # true?
docker stats CONTAINER                                         # watch usage climb
```

Fix: either the application has a memory leak (profile it), or the `--memory` limit (Chapter 11)/Kubernetes memory limits (Chapter 15) is set too low for genuine peak usage — measure actual usage under realistic load and set limits with appropriate headroom, and ensure requests/limits are tuned separately from arbitrary defaults.

17b.10 Swarm/Kubernetes Deployment Stuck (“Pending” or Not Updating)

```
docker service ps SERVICE --no-trunc      # Swarm: see task errors
kubectl describe pod POD                   # Kubernetes: see Events section
                                           # at the bottom of the output -
                                           # this almost always explains why
```

Common causes: insufficient cluster resources for the requested replicas/resource limits; an image pull failure (wrong tag, missing registry credentials — check `imagePullSecrets` in Kubernetes); a failing readiness/liveness probe preventing the rollout from completing (Chapters 14, 15); placement constraints that no node satisfies.

Chapter 18: Conclusion and Next Steps

18.1 What You’ve Learned

Across this course, you’ve moved from first principles to production practice:

- **Why containers exist** — the “works on my machine” problem, and how namespaces and cgroups give containers VM-like isolation at a fraction of the overhead (Chapter 1).
- **How Docker is architected** — the client-server model, the layered image filesystem, and the containerd/runc chain underneath the daemon (Chapters 2, 11).
- **The full CLI toolkit** — running, inspecting, debugging, and cleaning up containers and images (Chapter 4).
- **Configuration** via environment variables, following twelve-factor principles (Chapter 5).
- **Building images** with Dockerfiles, layer caching, multi-stage builds, and the CMD/ENTRYPOINT distinction (Chapter 6).
- **Networking** containers together and to the outside world, and why user-defined bridge networks matter (Chapter 7).
- **Persisting data** correctly with volumes, bind mounts, and tmpfs (Chapter 8).
- **Orchestrating multi-container apps on one host** with Docker Compose (Chapter 9).
- **Distributing images** through registries, public and private (Chapter 10).
- **Platform nuances** on Windows and macOS (Chapter 12).
- **Scaling beyond one host** with Docker Swarm and Kubernetes, including the shared vocabulary of services, self-healing, and rolling updates that underlies both (Chapters 13–15).
- **Shipping code to production safely** through a full CI/CD pipeline, deployment strategies, and observability practices (Chapter 16).
- **Securing and hardening** containerized systems for real-world use (Chapter 17).

18.2 Suggested Learning Path Going Forward

1. **Practice daily-driver Docker usage** — containerize a personal project end-to-end: write a Dockerfile, add a Compose file for its dependencies (database, cache), and get comfortable with the full CLI from muscle memory.
2. **Get hands-on with Kubernetes** — spin up a local cluster (`kind` or `minikube`), deploy the same application you containerized above, and work through Deployments, Services, and Ingress yourself.
3. **Build a real CI/CD pipeline** — wire up GitHub Actions (or GitLab CI) to build, test, and push an image on every commit, then deploy it automatically to a small server, Swarm, or a managed Kubernetes cluster (EKS/GKE/AKS all offer free-tier or low-cost entry points).
4. **Study a managed Kubernetes offering** in depth — most production Kubernetes today runs on a cloud provider’s managed control plane, which handles much of the operational complexity described in Chapter 15.
5. **Explore the wider cloud-native ecosystem** — Helm (Chapter 15), service meshes (Istio, Linkerd) for advanced traffic management and mTLS between services, GitOps tools (ArgoCD, Flux), and observability stacks (Prometheus/Grafana).
6. **Deepen container security knowledge** — explore image scanning tooling (Trivy), runtime security monitoring (Falco), and the CIS Docker/Kubernetes Benchmarks for hardening guidance.

18.3 Closing Thought

Containers, and the DevOps culture that grew alongside them, exist to solve one underlying problem: making it fast, safe, and repeatable to get software from a developer's editor into the hands of real users. Every concept in this course — layered images, isolated networks, declarative orchestration, automated pipelines — is ultimately in service of that goal. The tools will keep evolving, but the underlying problem, and the principles used to solve it, will remain the foundation worth understanding deeply.

Appendix A: Complete Command Cheat Sheet

A.1 Container Lifecycle

```
docker run [OPTIONS] IMAGE [CMD]      # create + start a container
docker create IMAGE                    # create without starting
docker start CONTAINER                 # start an existing container
docker stop CONTAINER                  # graceful stop (SIGTERM then SIGKILL)
docker restart CONTAINER               # stop then start
docker pause / unpause CONTAINER       # freeze / resume processes
docker kill CONTAINER                  # immediate SIGKILL
docker rm CONTAINER                    # remove a stopped container
docker rm -f CONTAINER                 # force-remove (stop + remove)
```

A.2 Inspection and Debugging

```
docker ps [-a] [-q]                   # list containers
docker inspect CONTAINER|IMAGE         # full JSON metadata
docker logs [-f] [--tail N] CONTAINER  # view logs
docker exec -it CONTAINER sh|bash      # shell into a running container
docker top CONTAINER                   # processes inside a container
docker stats [CONTAINER]               # live resource usage
docker cp SRC DEST                     # copy files to/from a container
docker diff CONTAINER                  # files changed vs. the image
```

A.3 Images

```
docker images / docker image ls       # list local images
docker pull IMAGE[:TAG]                # download from a registry
docker push IMAGE[:TAG]                # upload to a registry
docker build -t NAME:TAG PATH           # build from a Dockerfile
docker tag SRC TARGET                  # add a tag/alias
docker rmi IMAGE / docker image rm IMAGE # delete a local image
docker history IMAGE                   # view layer history
```

A.4 Networks

```
docker network ls / create / inspect / rm
docker network connect NETWORK CONTAINER
docker network disconnect NETWORK CONTAINER
```

A.5 Volumes

```
docker volume ls / create / inspect / rm
docker run -v NAME:/path IMAGE         # named volume
docker run -v /host/path:/path IMAGE   # bind mount
```

```
docker run --mount type=volume,source=NAME,target=/path IMAGE
```

A.6 Cleanup

```
docker system df           # disk usage summary
docker system prune [-a] [--volumes] # remove unused objects
docker container prune     # stopped containers only
docker image prune [-a]    # dangling (or all unused) images
docker volume prune       # unused volumes
docker network prune      # unused networks
```

A.7 Compose

```
docker compose up [-d] [--build]
docker compose down [--volumes]
docker compose ps / logs [-f] SERVICE
docker compose exec SERVICE sh
docker compose build / stop / start / restart SERVICE
docker compose config
```

A.8 Swarm

```
docker swarm init / join / leave
docker node ls / inspect / update
docker service create / ls / ps / update / scale / logs
docker stack deploy -c FILE STACK
docker stack ls / services / ps / rm
docker secret create / ls
```

A.9 Kubernetes (kubectl)

```
kubectl get pods,deploy,svc,nodes
kubectl describe pod NAME
kubectl logs [-f] POD [-c CONTAINER]
kubectl exec -it POD -- sh
kubectl apply -f FILE.yaml
kubectl delete -f FILE.yaml
kubectl scale deployment NAME --replicas=N
kubectl set image deployment/NAME CONTAINER=IMAGE
kubectl rollout status / history / undo deployment/NAME
kubectl get events --sort-by=.metadata.creationTimestamp
```

Appendix B: Dockerfile Instruction Reference

Instruction	Purpose	Example
FROM	Base image	FROM node:20-alpine
LABEL	Metadata	LABEL version="1.0"
ARG	Build-time variable	ARG BUILD_ENV=production
ENV	Runtime environment variable	ENV PORT=8080
WORKDIR	Working directory	WORKDIR /app
COPY	Copy files into image	COPY . .
ADD	Copy + extract/fetch	ADD app.tar.gz /app
RUN	Execute a build-time command	RUN npm install
EXPOSE	Document listening port	EXPOSE 8080
VOLUME	Declare a mount point	VOLUME /data
USER	Set user for subsequent instructions	USER appuser
ENTRYPOINT	Fixed executable	ENTRYPOINT ["python","app.py"]
CMD	Default args / command	CMD ["--port","8080"]
HEALTHCHECK	Container health test	HEALTHCHECK CMD curl -f localhost/health
STOPSIGNAL	Signal used to stop container	STOPSIGNAL SIGTERM
ONBUILD	Deferred instruction for child builds	ONBUILD COPY . /app
SHELL	Change default shell for shell-form instructions	SHELL ["/bin/bash","-c"]

Appendix C: Glossary

Bind mount — a mount of an exact host filesystem path into a container.

Bridge network — the default Docker networking mode, creating an isolated private network on a single host.

Build context — the set of files sent to the Docker daemon when building an image.

cgroups (control groups) — a Linux kernel feature for limiting and accounting for resource usage (CPU, memory, I/O) per process group.

CMD — Dockerfile instruction specifying the default command/arguments for a container.

Container — a running (or stopped) instance of an image, isolated via namespaces and cgroups.

containerd — a high-level container runtime managing image transfer, storage, and container execution, used by both Docker and Kubernetes.

Deployment — a Kubernetes resource that manages a replicated, self-healing set of Pods.

Digest — a content-addressable SHA-256 hash uniquely identifying an image’s exact content.

Dockerfile — a text file of instructions for building a Docker image.

ENTRYPOINT — Dockerfile instruction specifying the fixed executable a container always runs.

Image — an immutable, layered filesystem template used to create containers.

Ingress — a Kubernetes resource for HTTP(S) routing into the cluster.

Kubernetes (k8s) — an open-source container orchestration platform, the industry-standard for running containers at scale.

Layer — one read-only filesystem “slice” that makes up a Docker image, typically corresponding to one Dockerfile instruction.

Namespace (Linux) — a kernel feature providing isolated views of system resources (PIDs, network, mounts, etc.) per process.

Namespace (Kubernetes) — a mechanism for partitioning a cluster into virtual sub-clusters.

OCI (Open Container Initiative) — the organization defining standard specifications for container images and runtimes.

Orchestrator — a system that automates the deployment, scaling, networking, and management of containerized applications across a cluster of machines.

Overlay network — a network driver connecting containers across multiple Docker hosts, used by Docker Swarm.

Pod — the smallest deployable unit in Kubernetes; one or more containers sharing network and storage.

Registry — a service for storing and distributing container images (e.g., Docker Hub).

Rolling update — a deployment strategy that replaces old container instances with new ones incrementally, avoiding downtime.

runc — a low-level, OCI-compliant runtime that performs the actual work of creating and starting a container process.

Service (Swarm/Kubernetes) — an abstraction over a group of replicated container instances, providing stable networking and desired-state management.

Twelve-Factor App — a methodology for building software-as-a-service applications, emphasizing (among other things) storing configuration in the environment.

Volume — Docker-managed persistent storage, decoupled from any single container's lifecycle.

WSL2 — Windows Subsystem for Linux version 2; runs a real Linux kernel on Windows, used as Docker Desktop's default backend.

Appendix E: Further Resources

E.1 Official Documentation

- Docker Docs — docs.docker.com — the canonical reference for CLI commands, Dockerfile syntax, and Compose file schema.
- Kubernetes Docs — kubernetes.io/docs — comprehensive, including an interactive tutorial environment.
- OCI Specifications — opencontainers.org — the underlying image and runtime specs referenced throughout Chapter 11.

E.2 Hands-On Practice

- Play with Docker (labs.play-with-docker.com) — free, temporary, browser-based Docker (and Swarm) environments.
- Killercodea / Katacodea-style interactive Kubernetes scenarios — browser-based `kubect1` practice without installing anything.
- `kind` / `minikube` / `k3d` — local, disposable Kubernetes clusters for hands-on practice (Chapter 15).

E.3 Reference Images and Examples

- Docker Hub Official Images — hub.docker.com — study the Dockerfiles of well-maintained official images (postgres, nginx, node) for real-world best practices.
- The Twelve-Factor App — 12factor.net — the methodology referenced in Chapter 5 for environment-driven configuration.

E.4 Security

- CIS Docker Benchmark and CIS Kubernetes Benchmark — widely used hardening checklists.
- Trivy, Docker Scout, Gype — open-source and built-in vulnerability scanners referenced in Chapters 10 and 17.
- Falco — open-source runtime security monitoring for containers.

E.5 Suggested Practice Progression

1. Containerize a small personal project with a single Dockerfile.
2. Add a second service (a database) and connect them with Docker Compose.
3. Push your image to a registry and pull it on a second machine (or a cloud VM) to confirm true portability.
4. Set up a CI pipeline that builds, tests, and pushes automatically on every commit.
5. Deploy the same application to a local Kubernetes cluster (`kind/minikube`), replicating what Compose did, using Deployments and Services.
6. Add a readiness/liveness probe and deliberately break your application to watch Kubernetes' self-healing behavior in action.
7. Add resource limits, and load-test to see what happens when you exceed them.

Appendix D: Review Questions and Answers

Use these questions to self-check your understanding of each chapter before moving on. Answers are provided immediately below each question for quick review.

D.1 Chapter 1 — Introduction to DevOps and Containerization

Q1. What kernel features does Docker rely on to isolate containers? A. Namespaces (isolation of PIDs, network, mounts, etc.) and cgroups (resource limiting and accounting).

Q2. Why is a container image typically megabytes while a VM image is typically gigabytes? A. A container does not bundle a guest operating system; it shares the host's kernel and only packages the application and its specific dependencies. A VM bundles an entire guest OS.

Q3. What organization standardized the container image and runtime formats, and why does that matter? A. The Open Container Initiative (OCI). It ensures images and runtimes from different vendors (Docker, containerd, Kubernetes, Podman) remain interoperable.

Q4. Name two DevOps principles besides automation. A. Continuous Integration/Continuous Delivery, and Infrastructure as Code (also acceptable: monitoring/observability, shared ownership).

D.2 Chapter 2 — Docker Architecture

Q1. What are the three major components of Docker's client-server architecture? A. The Docker client, the Docker daemon (`dockerd`), and a registry.

Q2. What is the relationship between `dockerd`, `containerd`, and `runc`? A. `dockerd` delegates container lifecycle management to `containerd`, which in turn uses `runc` (via a shim) to actually create and start the container process using namespaces and cgroups.

Q3. Why can a container's writable layer be deleted safely when the container is removed, without affecting the image it was created from? A. Because the container's writable layer sits on top of the image's read-only layers using copy-on-write; the image layers are never modified, only referenced.

D.3 Chapter 3 — Installing Docker

Q1. Why does Docker Desktop on macOS run a Linux virtual machine? A. Because containers require a Linux kernel, and macOS's kernel (XNU) cannot run Linux containers natively.

Q2. What is the recommended Windows backend for Docker Desktop today, and why? A. WSL2, because it runs a real, lightweight Linux kernel with near-native performance, replacing the heavier legacy Hyper-V backend.

Q3. Why should you avoid running Docker commands with `sudo` on a day-to-day basis? A. You shouldn't need to once your user is added to the `docker` group — though note that group membership is effectively equivalent to root access on the host.

D.4 Chapter 4 — The Docker CLI

Q1. What's the difference between `docker stop` and `docker rm`? A. `stop` halts a running container but keeps it (and its writable layer) on disk; `rm` permanently deletes a (stopped) container.

Q2. What's the difference between `docker run` and `docker exec`? A. `run` creates a brand-new container from an image; `exec` runs an additional process inside an already-running container.

Q3. Why does `docker run ubuntu bash` (without `-it`) typically exit immediately? A. Without an allocated TTY and open STDIN, the interactive shell process has no input to process and exits right away.

D.5 Chapter 5 — Environment Variables

Q1. What takes precedence: an ENV default set in a Dockerfile, or a value passed with `docker run -e`? A. The runtime `-e` value overrides the Dockerfile's ENV default.

Q2. Why shouldn't secrets be baked into a Dockerfile via ENV? A. ENV values become part of the image itself, visible to anyone who can pull the image or inspect its history — they should be supplied at runtime or via a secrets manager instead.

D.6 Chapter 6 — Images and the Dockerfile

Q1. Why does copying `package.json` and running `npm install` before copying the rest of the source code speed up rebuilds? A. Docker's build cache invalidates an instruction (and everything after it) only when its own inputs change; keeping the (slow) dependency-install step before the (frequently changing) source-code copy means it's cached and skipped unless dependencies actually change.

Q2. What is the key advantage of a multi-stage build? A. It allows a final image to contain only the runtime artifacts it needs (e.g., compiled output), leaving build tools, source code, and intermediate files out of the production image — reducing size and attack surface.

Q3. If a Dockerfile has `ENTRYPOINT ["python", "app.py"]` and `CMD ["--debug"]`, what happens when you run `docker run myimage --port 9000`? A. It runs `python app.py --port 9000` — the arguments given at `docker run` replace CMD but are still appended to the fixed ENTRYPOINT.

D.7 Chapter 7 — Networking

Q1. Why can't two containers on the default bridge network reach each other by container name? A. The default bridge network does not run Docker's embedded DNS server; only user-defined bridge networks provide automatic name-based service discovery.

Q2. Does `EXPOSE 8080` in a Dockerfile actually make port 8080 reachable from the host? A. No — EXPOSE is documentation only. The port must be published with `-p/-P` at `docker run` time (or `ports:` in Compose) to actually be reachable.

D.8 Chapter 8 — Storage

Q1. When should you use a bind mount instead of a named volume? A. When you need to reference a specific, existing path on the host filesystem directly — most commonly for live-editing source code during local development.

Q2. What happens to data in a `tmpfs` mount when the container stops? A. It's gone — `tmpfs` mounts live only in host RAM and are never written to disk.

D.9 Chapter 9 — Docker Compose

Q1. Does `depends_on` alone guarantee that a dependency is fully ready to accept connections? A. No — by default it only guarantees start order. Use `depends_on: condition: service_healthy` combined with a `healthcheck` on the dependency for actual readiness.

Q2. What command brings an entire multi-service application down, including its named volumes? A. `docker compose down --volumes` (plain `docker compose down` removes containers and networks but preserves named volumes by default).

D.10 Chapter 10 — Registries

Q1. Why might a production deployment reference an image by digest instead of by tag? A. Tags are mutable and can be repointed to different content over time; a digest uniquely and immutably identifies exact image content, guaranteeing reproducibility.

D.11 Chapter 11 — The Docker Engine

Q1. What Linux kernel feature does the default `overlay2` storage driver build on? A. OverlayFS, which implements the layered, copy-on-write filesystem used for image layers and container writable layers.

Q2. Name one benefit BuildKit provides over the legacy Docker builder. A. Parallel execution of independent build stages, better cache import/export (including to/from a registry), and build-time secrets that never end up baked into image layers (any one is acceptable).

D.12 Chapter 12 — Windows and macOS

Q1. Why is file I/O for bind-mounted directories often slower on Docker Desktop than on native Linux? A. Because the mounted directory must cross the boundary between the host OS filesystem and the Linux VM (or WSL2) running the actual container, incurring overhead not present when everything runs natively on Linux.

D.13 Chapter 13 — Orchestration Overview

Q1. What does “desired-state reconciliation” mean? A. You declare what you want (e.g., “5 replicas running”), and the orchestrator continuously and automatically works to make the actual cluster state match that desired state, correcting any drift such as a crashed container.

D.14 Chapter 14 — Docker Swarm

Q1. Why is an odd number of manager nodes recommended for a production Swarm cluster? A. Manager nodes use Raft consensus, which requires a majority (quorum) to make decisions; an odd number avoids tie votes and maximizes fault tolerance for a given node count.

Q2. What does Swarm’s routing mesh do? A. It makes a published service port reachable from every node in the swarm, even nodes not running a replica of that service, transparently routing traffic to a node that is.

D.15 Chapter 15 — Kubernetes

Q1. What is the smallest deployable unit in Kubernetes, and how does it differ from a plain container? A. The Pod — a group of one or more containers that are always scheduled together, sharing the same network namespace and, optionally, storage volumes.

Q2. What’s the practical difference between a readiness probe and a liveness probe? A. A failed readiness probe removes a Pod from a Service’s load-balancing pool (traffic stops routing to it) without killing it; a failed liveness probe causes the kubelet to kill and restart the Pod.

Q3. Why doesn’t modern Kubernetes use the full Docker Engine directly anymore? A. Since the removal of “dockershim” in Kubernetes 1.24+, Kubernetes talks directly to an OCI-compliant runtime like containerd or CRI-O via the Container Runtime Interface — Docker-built images remain fully compatible since they follow the standardized OCI image format.

D.16 Chapter 16 — CI/CD and Deployment

Q1. Why tag CI-built images with the Git commit SHA rather than only latest? A. It gives every build a unique, traceable, immutable identifier linking a deployed container directly back to the exact source commit it was built from.

Q2. What distinguishes a canary deployment from a rolling update? A. A canary deployment routes only a small percentage of traffic to the new version initially, gradually increasing it, to limit the blast radius of a bad release — a plain rolling update simply replaces instances incrementally without deliberately controlling traffic split.

D.17 Chapter 17 — Best Practices and Security

Q1. Why is `--privileged` discouraged for typical application containers? A. It grants nearly all Linux capabilities and direct device access, removing the isolation benefits containers are meant to provide; it should be reserved for specific infrastructure tooling that genuinely needs it.

Q2. What’s the risk of deleting a secret file in a later Dockerfile instruction after copying it in an earlier one? A. The file remains present in the earlier, still-cached image layer — deleting it in a later layer does not remove it from the image’s history, and it can still be extracted.